

MINI PROJECT BAYES_

Elvina

2023-12-29

Load library

```
library(coda)
library(rjags)
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
library(knitr)
library(purrr)
library(tidyr)
library(ggplot2)
```

set seed

```
set.seed(123)
```

Load Data

```
df <- read.csv('./ParisHousing.csv')
head(df)
```

##	squareMeters	numberOfRooms	hasYard	hasPool	floors	cityCode	cityPartRange	
## 1	75523	3	0	1	63	9373	3	
## 2	80771	39	1	1	98	39381	8	
## 3	55712	58	0	1	19	34457	6	
## 4	32316	47	0	0	6	27939	10	
## 5	70429	19	1	1	90	38045	3	
## 6	39223	36	0	1	17	39489	8	
##	numPrevOwners	made	isNewBuilt	hasStormProtector	basement	attic	garage	
## 1	8	2005	0		1	4313	9005	956
## 2	6	2015	1		0	3653	2436	128
## 3	8	2021	0		0	2937	8852	135
## 4	4	2012	0		1	659	7141	359
## 5	7	1990	1		0	8435	2429	292
## 6	6	2012	0		1	2009	4552	757
##	hasStorageRoom	hasGuestRoom	price					

```
## 1      0      7 7559082
## 2      1      2 8085990
## 3      1      9 5574642
## 4      0      3 3232561
## 5      1      4 7055052
## 6      0      1 3926647
```

Model 1

Set Dataframe as matrix and split dependent and independent variable

```
price <- as.matrix(df$price)
Y <- price

X <- df[, c("squareMeters", "numberOfRooms", "hasYard", "hasPool", "floors", "cityCode", "numP
revOwners", "made", "isNewBuilt", "hasStormProtector", "basement", "attic", "garage", "hasStor
ageRoom", "hasGuestRoom", "cityPartRange")]

names <- c("price", "Square Meters", "Number of Rooms", "Has Yard", "Has Pool", "Number of Floo
rs", "City Code", "Number of Previous Owners", "Year Made", "is Newly Built", "Has Storm Prote
ctor", "Basement Area", "Attic Area", "Garage Area", "Has Storage Room", "Has Guest Room", "Ci
ty Part Range")
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y, X=X, n=n, p=p)
params <- c("alpha", "beta")
burn <- 500
n.iter <- 2000
```

```
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- inprod(X[i,],beta[])
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Set Initial Value

```
inits = list()

inits$alpha = rnorm(1)
for(i in 1:p) {
  inits$beta[i] = rnorm(1)
}
inits$taue = 10
```

Compile Model

```
modell <- jags.model(model_string, data = data, n.chains=n.chains, quiet=TRUE, inits = inits)
```

Update Model

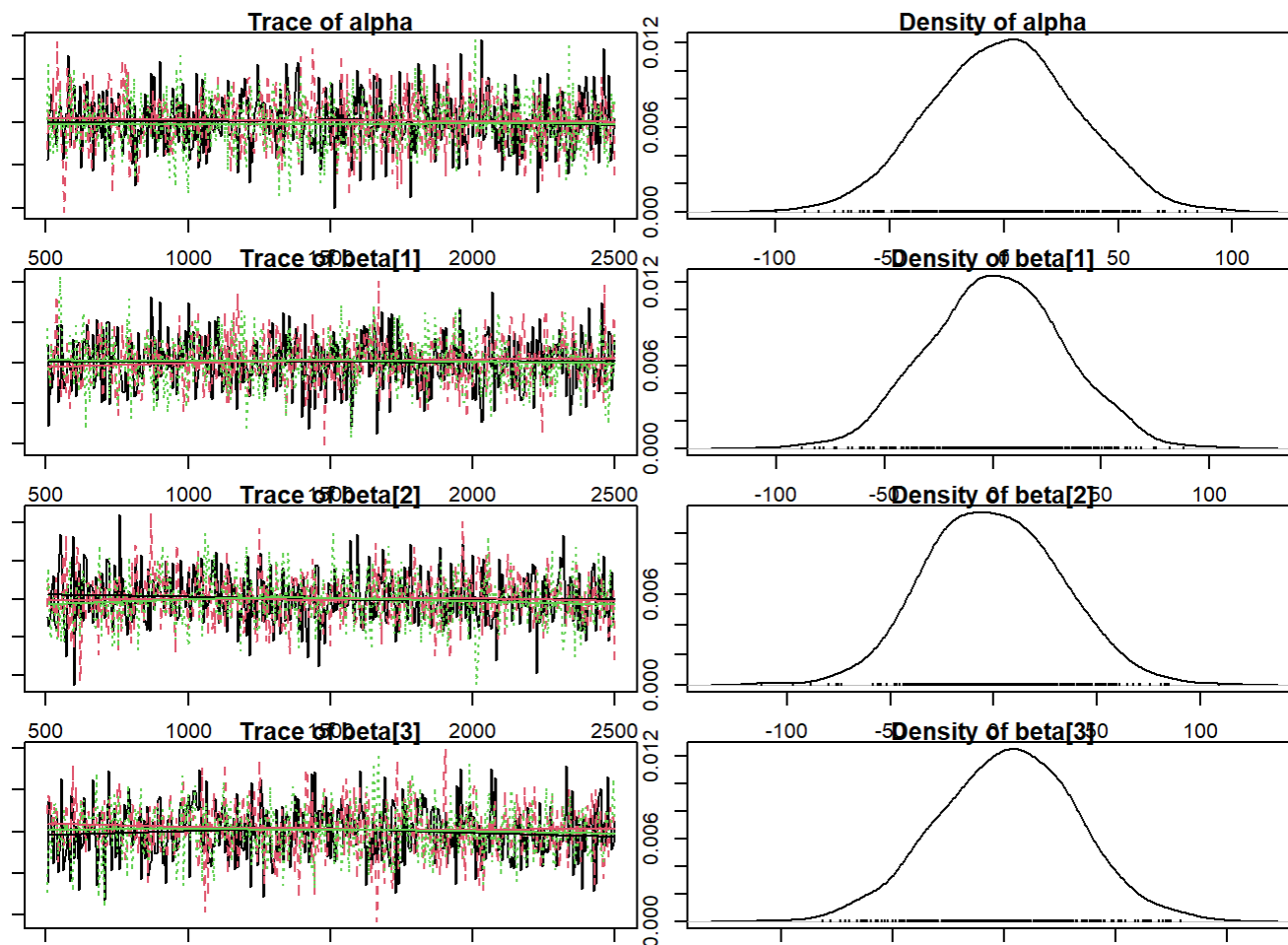
```
update(modell, burn)
```

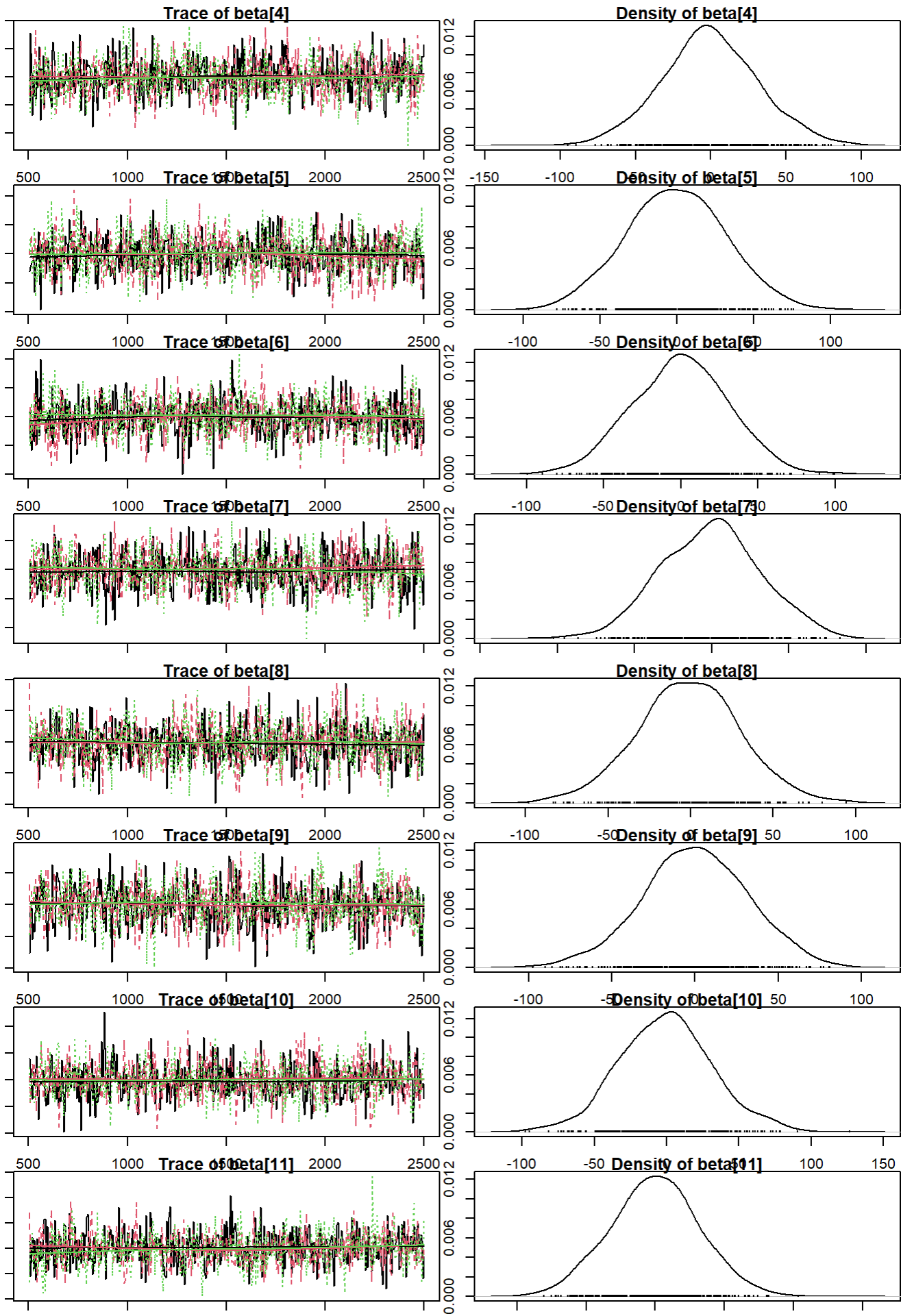
Get Posterior Samples from the model

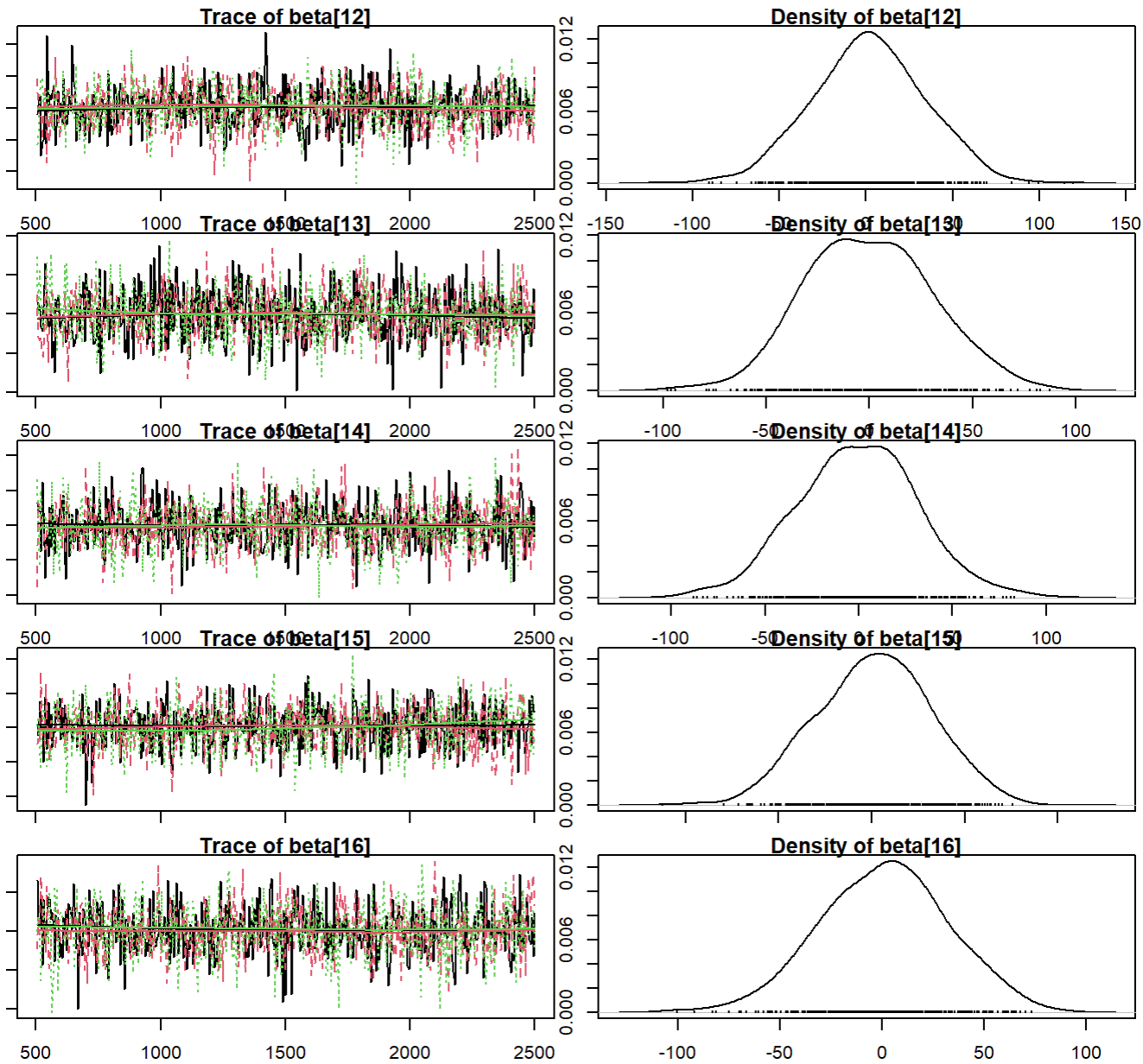
```
samples1 <- coda.samples(model1, variable.names=params, n.iter=n.iter, thin=thin)
```

Plot MCMC Chain Trace and Features Posterior Density

```
par(mar=c(1,1,1,1))
plot(samples1)
```







Descriptive Statistics of model1

summary(samples1)

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean      SD Naive SE Time-series SE
## alpha      -0.1109 31.65   0.9137          0.8701
## beta[1]     1.4739 31.05   0.8962          0.8968
## beta[2]    -0.3395 32.36   0.9340          0.9638
## beta[3]     1.1905 31.05   0.8963          0.8959
## beta[4]    -0.7156 31.72   0.9157          0.9156
## beta[5]    -0.5969 32.31   0.9326          0.9325
## beta[6]    -0.1771 31.13   0.8987          0.9184
## beta[7]    -0.7973 32.09   0.9262          0.9255
## beta[8]    -1.2257 31.15   0.8992          0.8995
## beta[9]     0.1952 31.48   0.9089          0.9347
## beta[10]   -0.6224 32.03   0.9246          0.8968
## beta[11]   -0.1120 32.06   0.9254          0.9251
## beta[12]    1.4060 32.89   0.9494          0.9204
## beta[13]   -0.2975 31.08   0.8971          0.8976
## beta[14]   -0.7652 32.43   0.9362          0.9367
## beta[15]    1.1699 30.86   0.8908          0.8912
## beta[16]    1.0599 31.48   0.9086          0.9091
##
## 2. Quantiles for each variable:
##
##              2.5%    25%      50%    75%  97.5%
## alpha      -61.68 -21.47   0.3241 20.56 59.50
## beta[1]    -57.20 -19.13   1.6712 22.60 61.57
## beta[2]    -60.42 -23.23 -0.9622 21.38 62.95
## beta[3]    -62.09 -19.21   2.4565 22.59 62.57
## beta[4]    -63.22 -21.06 -1.3499 20.25 62.93
## beta[5]    -62.46 -22.61 -0.8115 21.38 64.21
## beta[6]    -60.01 -21.97 -0.1892 20.56 58.44
## beta[7]    -61.14 -23.62   0.5906 20.59 61.59
## beta[8]    -62.16 -21.24 -1.2905 19.29 60.24
## beta[9]    -66.11 -20.26   0.6630 22.24 60.60
## beta[10]   -64.24 -21.77 -0.2163 20.12 66.65
## beta[11]   -60.66 -21.24   0.0317 20.48 62.23
## beta[12]   -61.66 -19.49   1.2401 22.96 63.13
## beta[13]   -57.46 -21.93 -0.6152 20.84 62.85
## beta[14]   -64.20 -22.06 -0.8384 20.30 62.88
## beta[15]   -59.43 -19.80   2.2784 22.26 59.21
## beta[16]   -63.28 -20.25   1.9391 22.39 61.36
```

Show Alpha and Betas to the corresponding Features

```
sum <- summary(samples1)
rownames(sum$statistics) <- names
rownames(sum$quantiles) <- names
sum$statistics <- round(sum$statistics,3)
sum$quantiles <- round(sum$quantiles,3)
sum
```

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##
```

	Mean	SD	Naive SE	Time-series SE
price	-0.111	31.65	0.914	0.870
Square Meters	1.474	31.05	0.896	0.897
Number of Rooms	-0.339	32.36	0.934	0.964
Has Yard	1.190	31.05	0.896	0.896
Has Pool	-0.716	31.72	0.916	0.916
Number of Floors	-0.597	32.31	0.933	0.932
City Code	-0.177	31.13	0.899	0.918
Number of Previous Owners	-0.797	32.09	0.926	0.926
Year Made	-1.226	31.15	0.899	0.900
is Newly Built	0.195	31.48	0.909	0.935
Has Storm Protector	-0.622	32.03	0.925	0.897
Basement Area	-0.112	32.06	0.925	0.925
Attic Area	1.406	32.89	0.949	0.920
Garage Area	-0.298	31.07	0.897	0.898
Has Storage Room	-0.765	32.43	0.936	0.937
Has Guest Room	1.170	30.86	0.891	0.891
City Part Range	1.060	31.48	0.909	0.909

```
##
## 2. Quantiles for each variable:
##
##
```

	2.5%	25%	50%	75%	97.5%
price	-61.68	-21.46	0.324	20.56	59.50
Square Meters	-57.20	-19.13	1.671	22.60	61.58
Number of Rooms	-60.42	-23.23	-0.962	21.38	62.95
Has Yard	-62.09	-19.21	2.456	22.59	62.57
Has Pool	-63.22	-21.06	-1.350	20.25	62.93
Number of Floors	-62.46	-22.61	-0.811	21.38	64.22
City Code	-60.02	-21.97	-0.189	20.56	58.44
Number of Previous Owners	-61.14	-23.62	0.591	20.59	61.59
Year Made	-62.16	-21.24	-1.290	19.29	60.24
is Newly Built	-66.11	-20.26	0.663	22.24	60.60

## Has Storm Protector	-64.24	-21.77	-0.216	20.12	66.65
## Basement Area	-60.66	-21.24	0.032	20.48	62.23
## Attic Area	-61.66	-19.49	1.240	22.96	63.13
## Garage Area	-57.46	-21.93	-0.615	20.84	62.85
## Has Storage Room	-64.20	-22.06	-0.838	20.30	62.88
## Has Guest Room	-59.43	-19.80	2.278	22.26	59.21
## City Part Range	-63.28	-20.25	1.939	22.39	61.36

Check Convergence

```
gelman.diag(samples1)
```

```
## Potential scale reduction factors:
##
##          Point est. Upper C.I.
## alpha          1.002      1.010
## beta[1]         1.001      1.006
## beta[2]         0.999      0.999
## beta[3]         1.000      1.000
## beta[4]         1.004      1.015
## beta[5]         1.004      1.012
## beta[6]         1.000      1.004
## beta[7]         1.003      1.012
## beta[8]         1.003      1.010
## beta[9]         1.006      1.026
## beta[10]        1.003      1.008
## beta[11]        1.003      1.007
## beta[12]        1.000      1.004
## beta[13]        1.001      1.006
## beta[14]        1.000      1.003
## beta[15]        0.999      1.001
## beta[16]        1.000      1.000
##
## Multivariate psrf
##
## 1.02
```

Compile results

```
ESS1 <- effectiveSize(samples1)
out1 <- summary(samples1)$quantiles
rownames(out1)<-names
ESS1
```

```
##      alpha  beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
## 1369.279 1200.000 1135.127 1200.000 1200.000 1200.000 1158.094 1200.000
##  beta[8]  beta[9]  beta[10] beta[11] beta[12] beta[13] beta[14] beta[15]
## 1200.000 1156.274 1295.456 1200.000 1293.756 1200.000 1200.000 1200.000
## beta[16]
```

1200.000

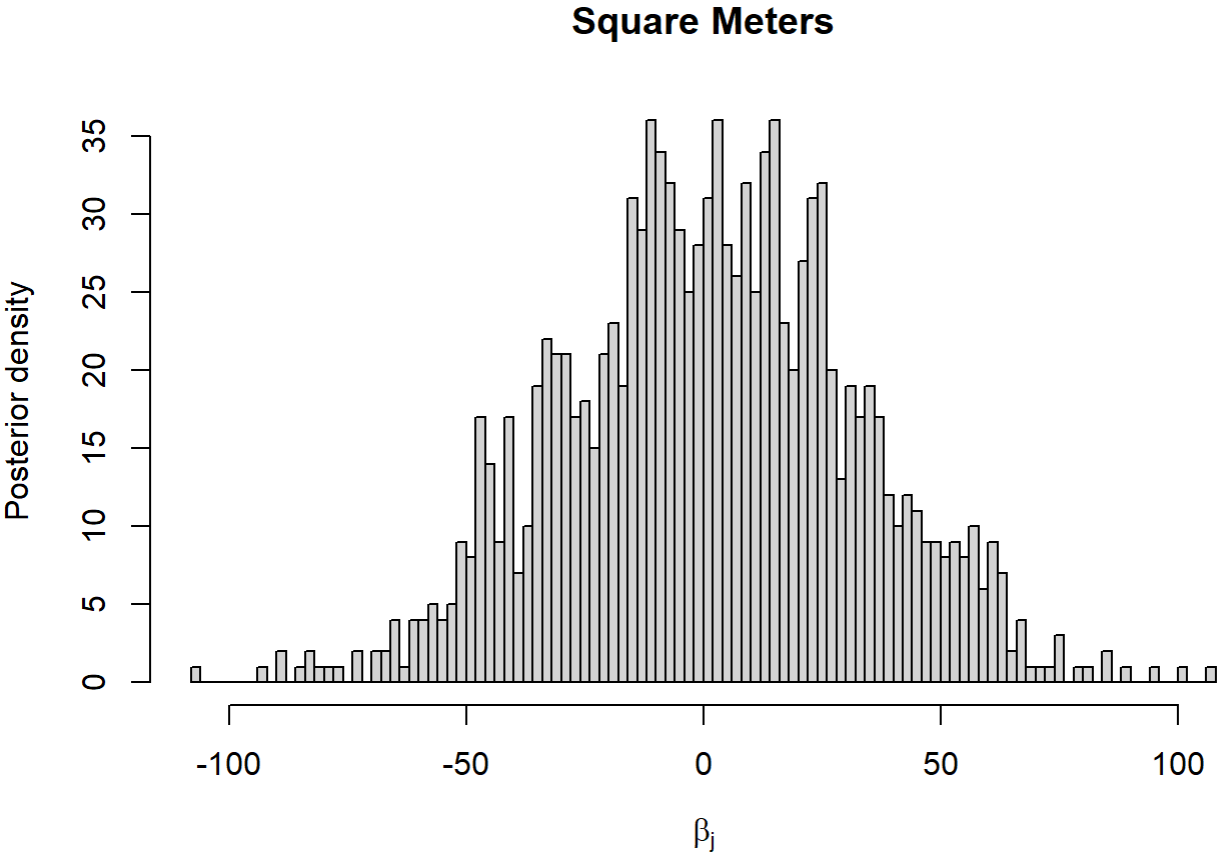
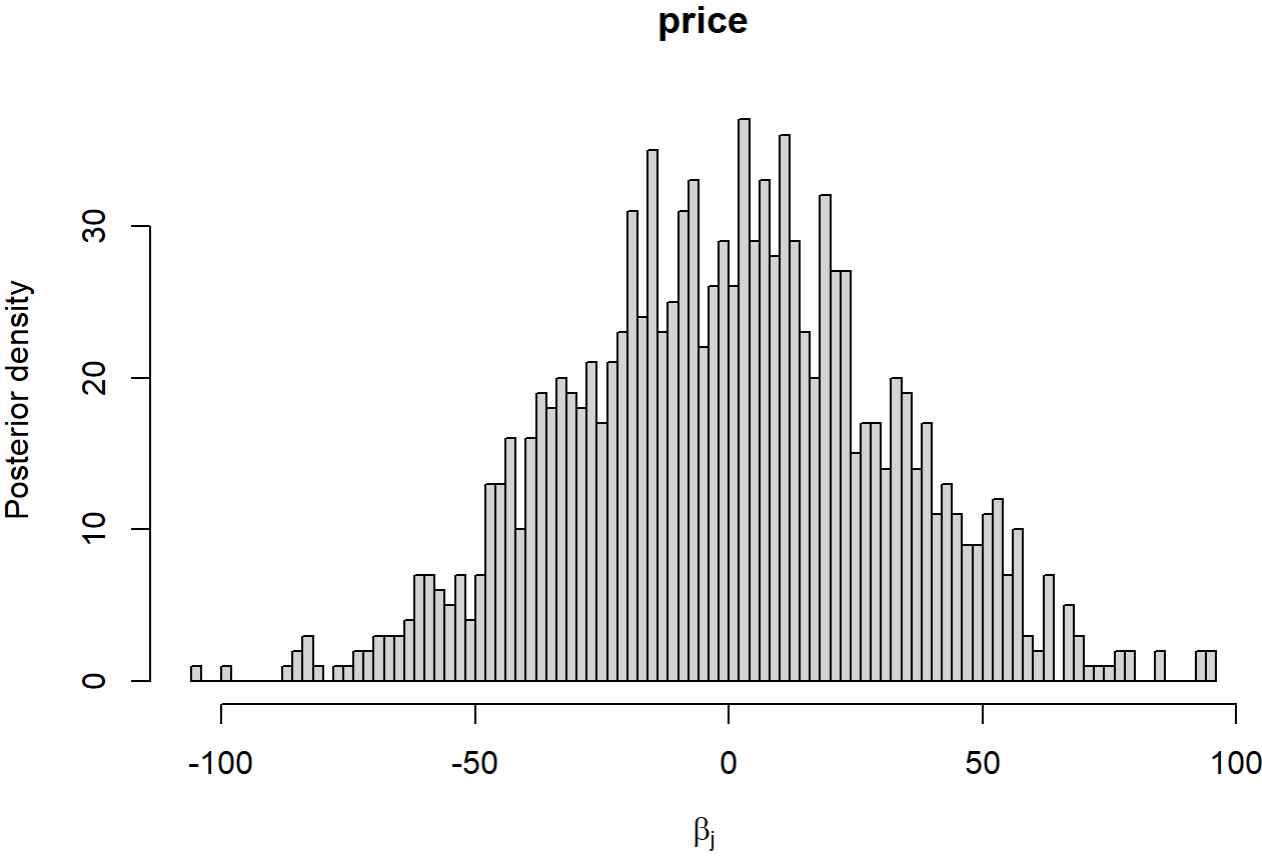
Compute DIC & WAIC

```
# DIC
dic1 <- dic.samples(model1, n.iter=n.iter)

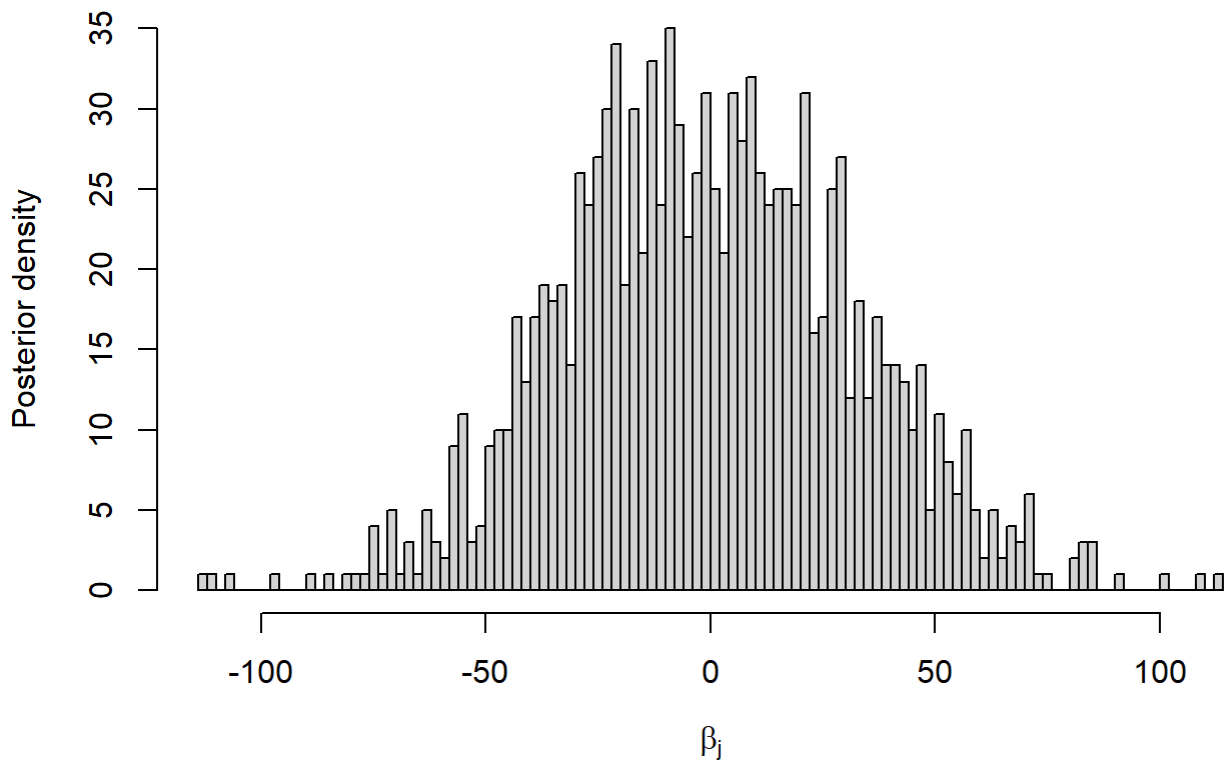
# WAIC
waic1 <- coda.samples(model1, variable.names=c("like"), n.iter=n.iter)
like1 <- waic1[[1]]
fbar1 <- colMeans(like1)
P1 <- sum(apply(log(like1), 2, var))
WAIC1 <- -2*sum(log(fbar1))+2*P1
```

Plot Posterior Density Individually

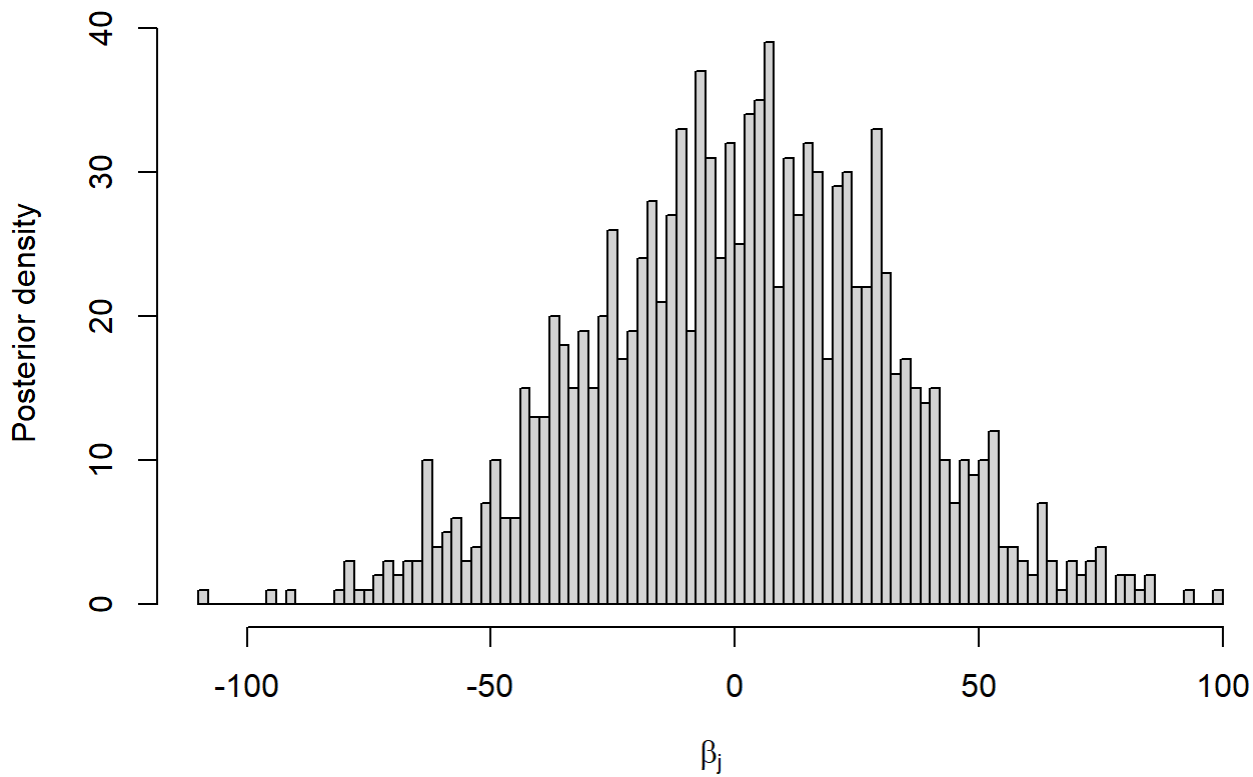
```
beta <- NULL
for(l in 1:n.chains){
  beta <- rbind(beta, samples1[[l]])
}
colnames(beta) <- names
for(j in 1:17){
  hist(beta[,j], xlab=expression(beta[j]), ylab="Posterior density",
  breaks=100, main=names[j])
}
```

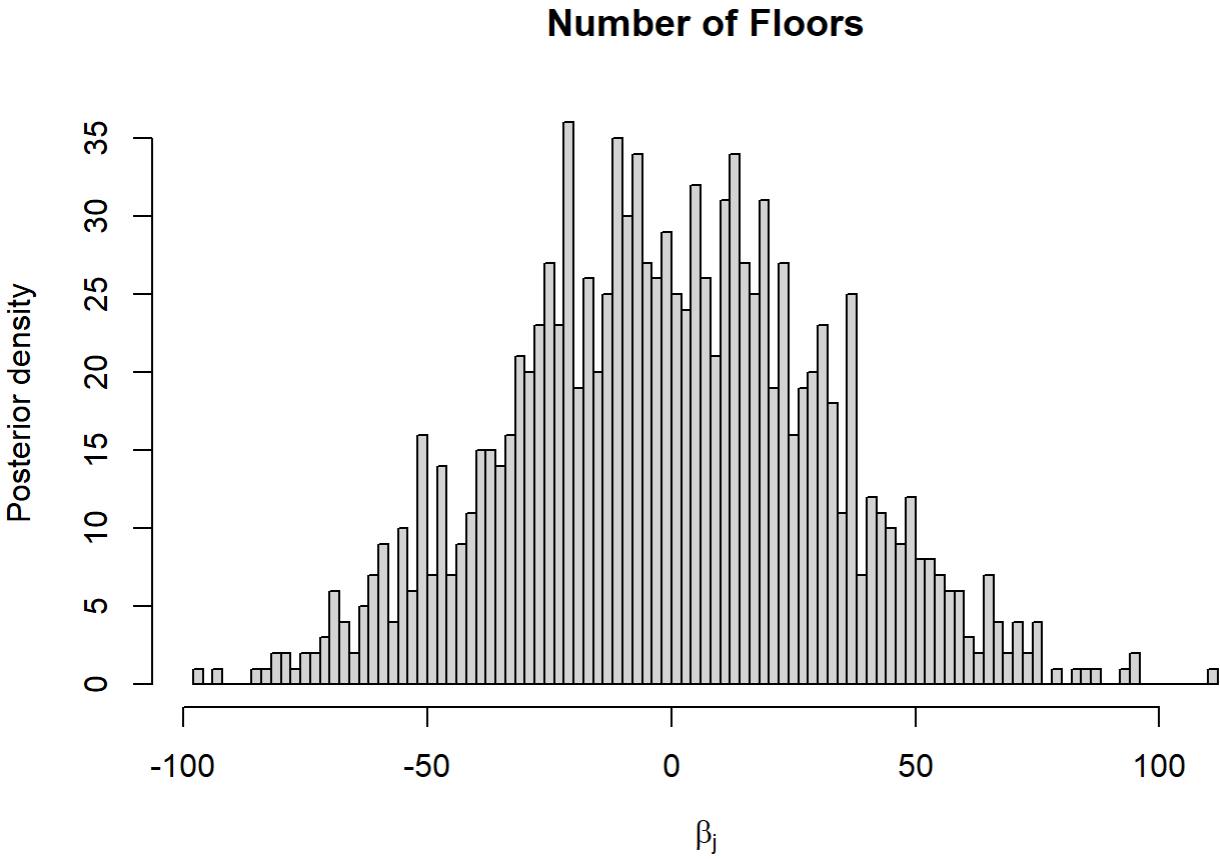
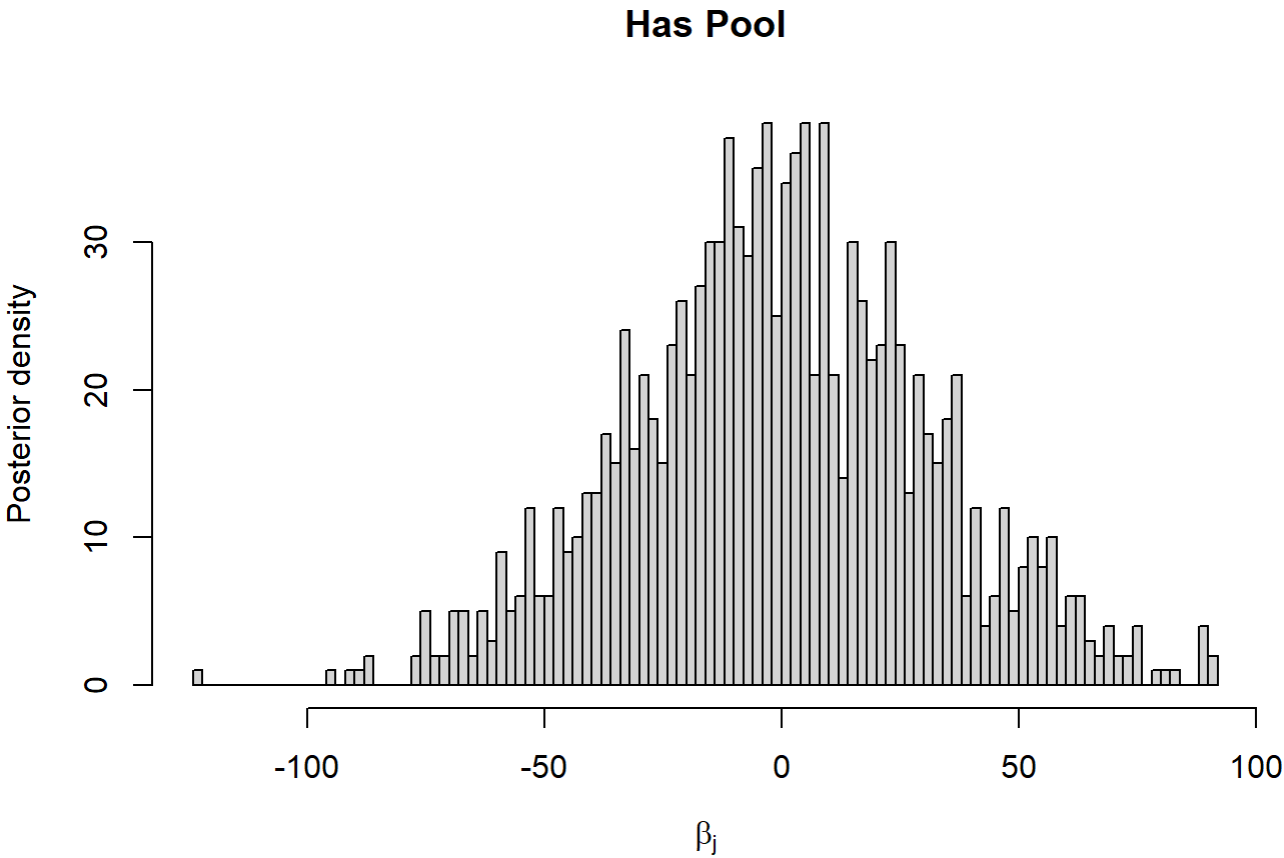


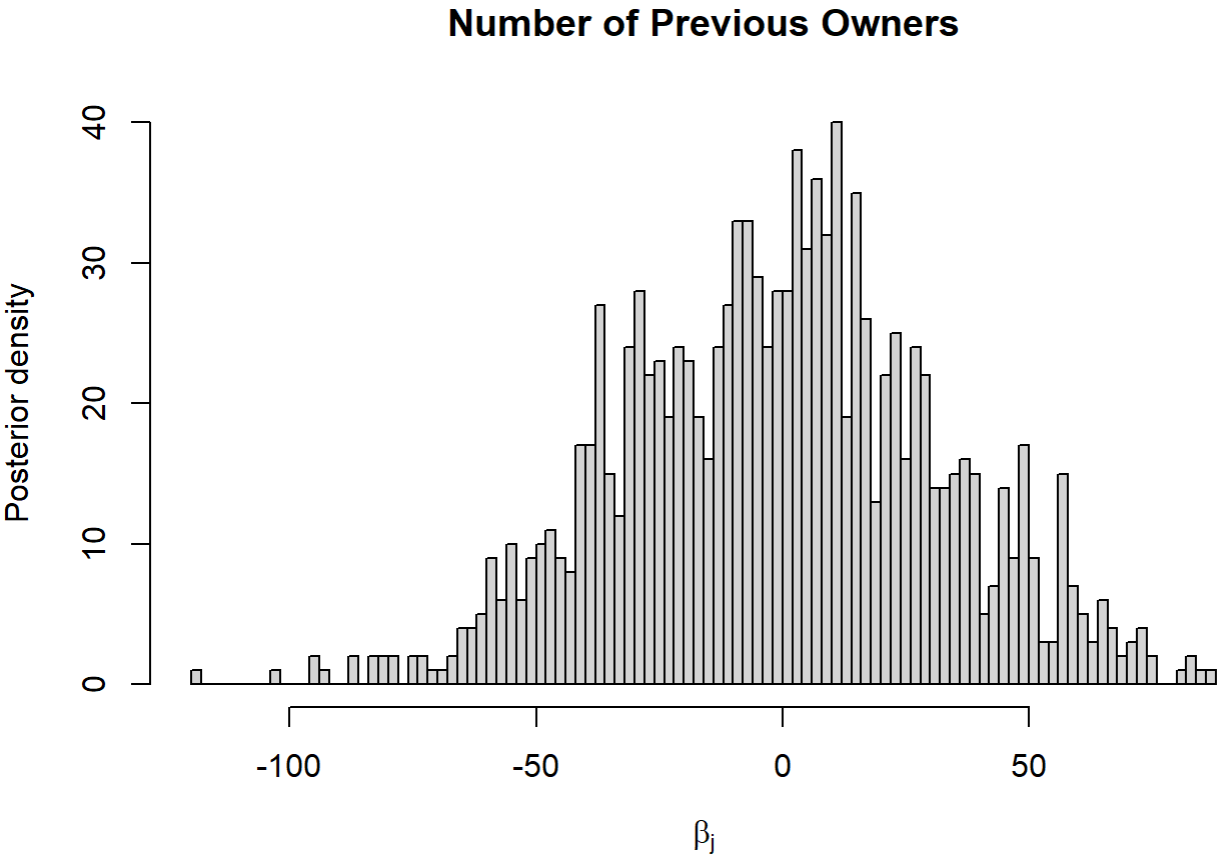
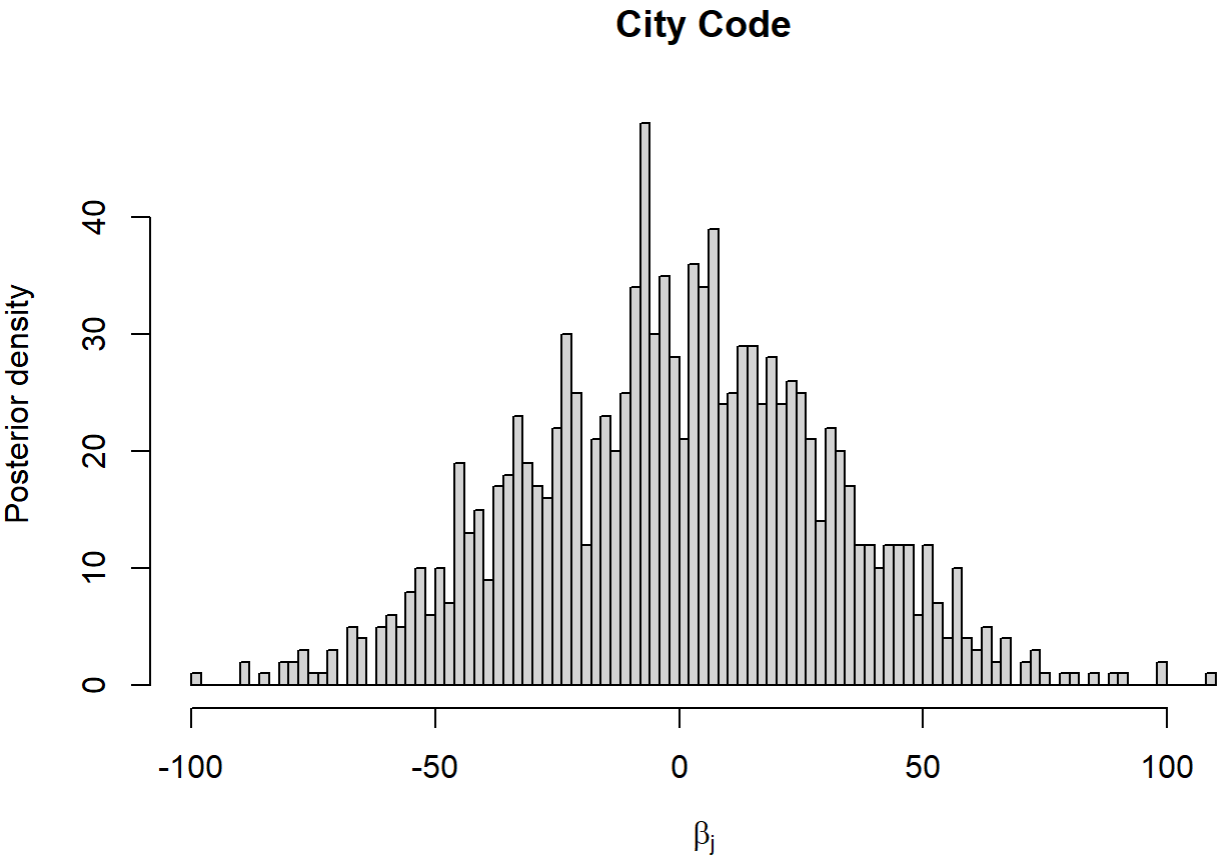
Number of Rooms

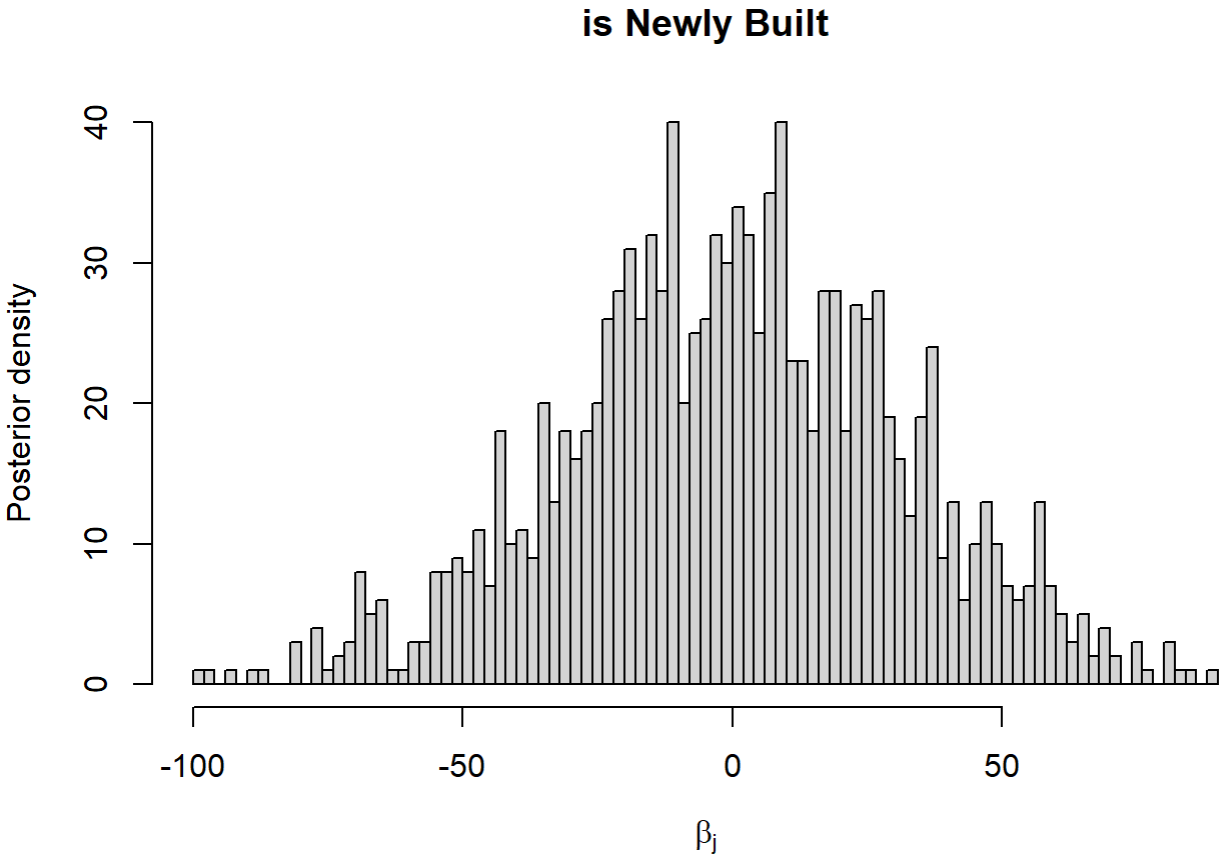
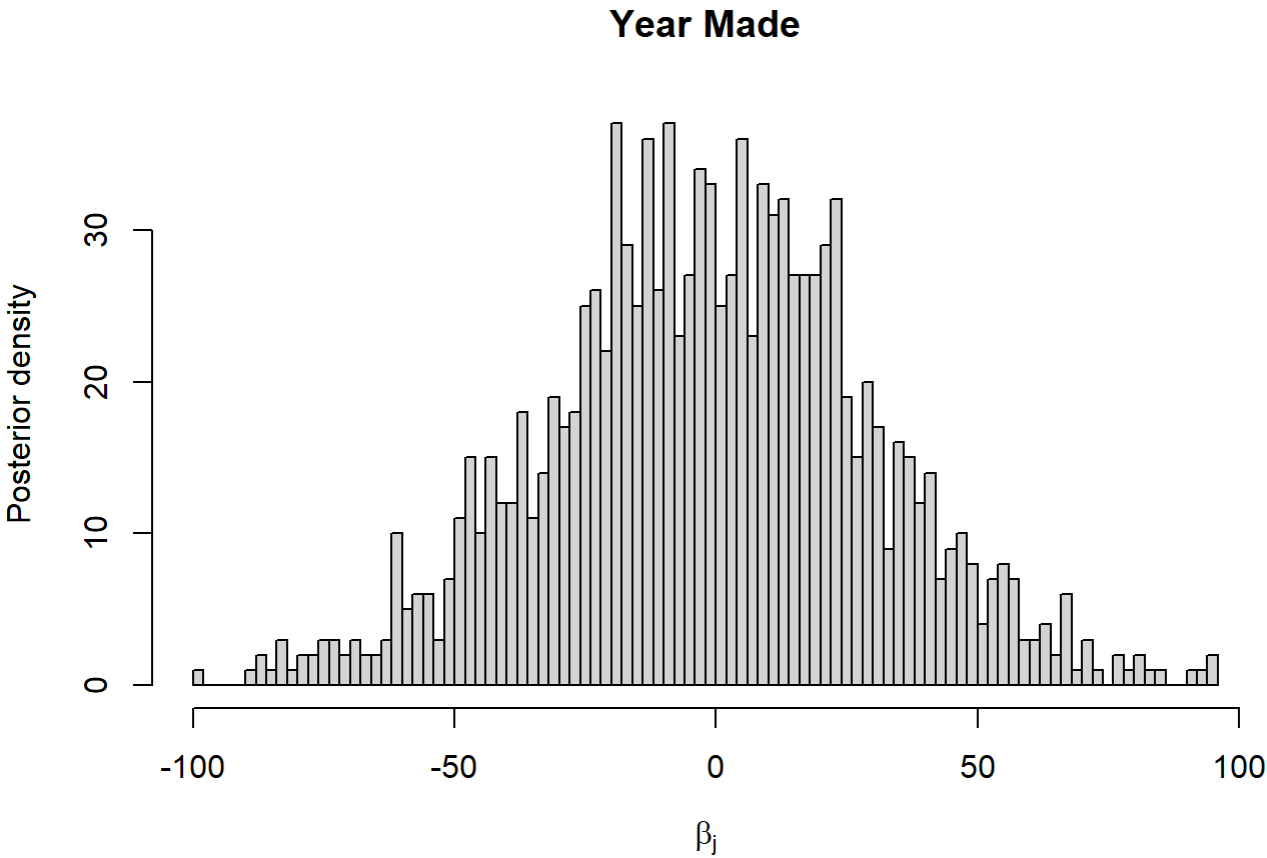


Has Yard

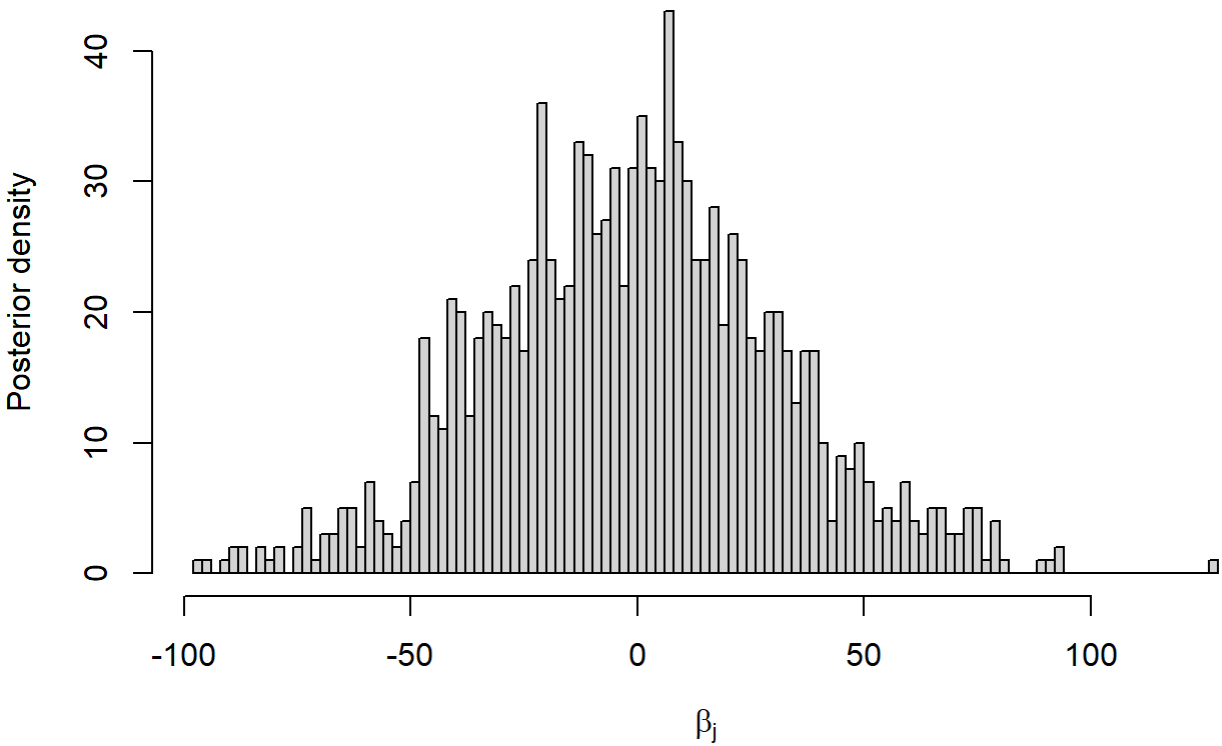




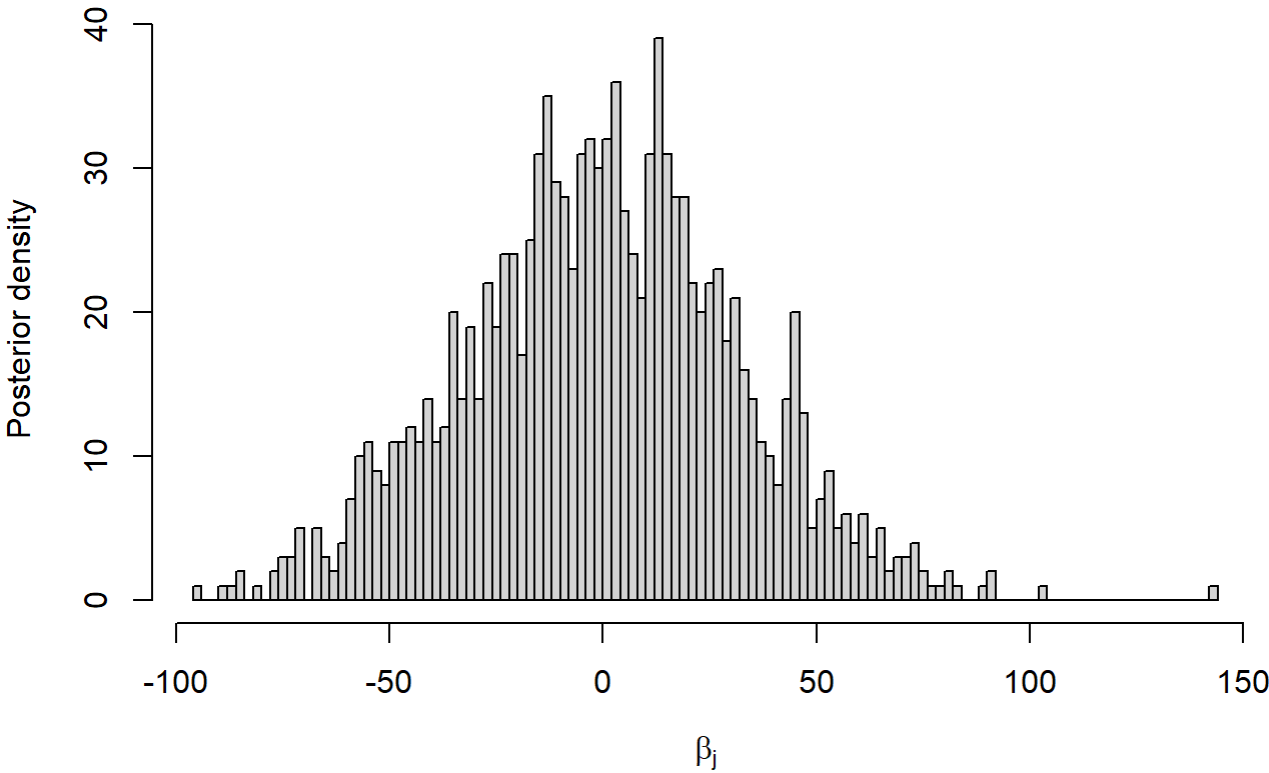


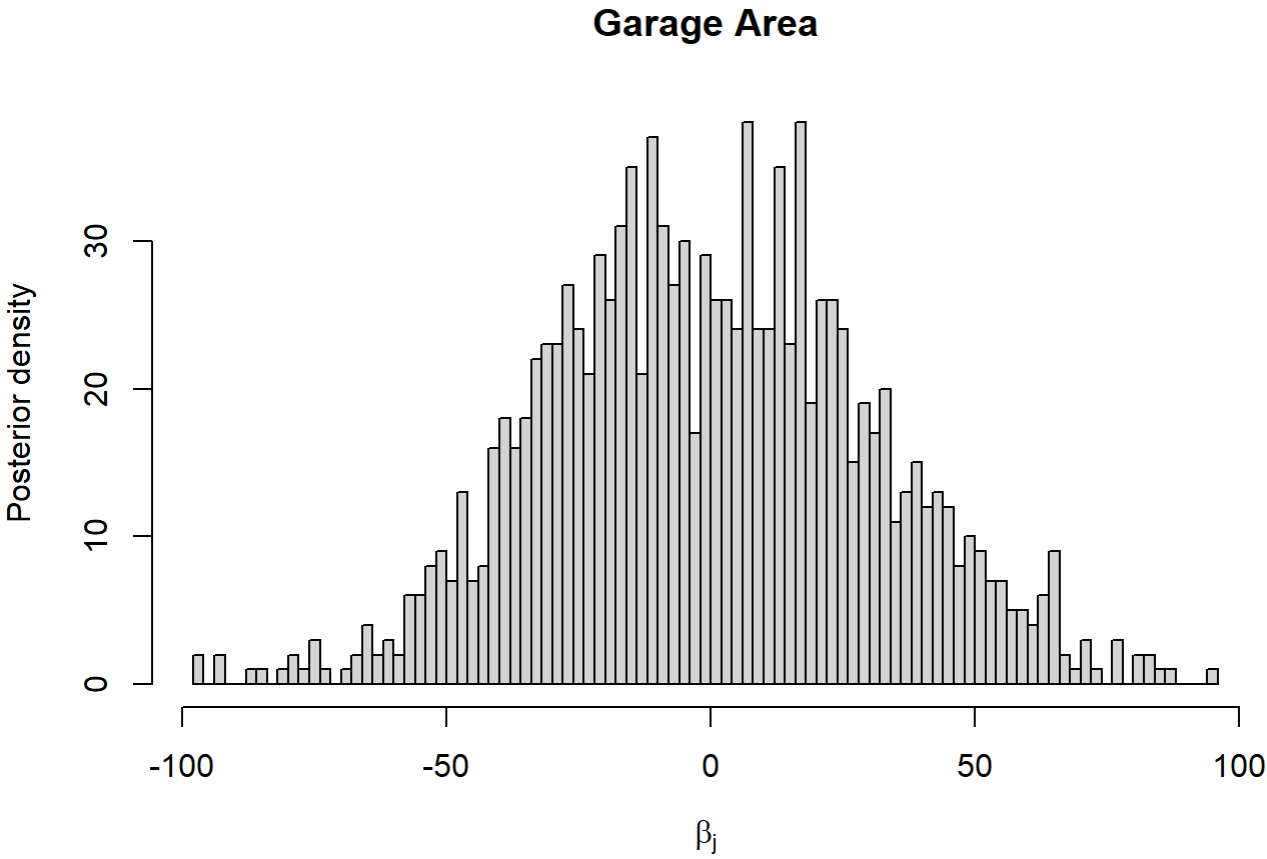
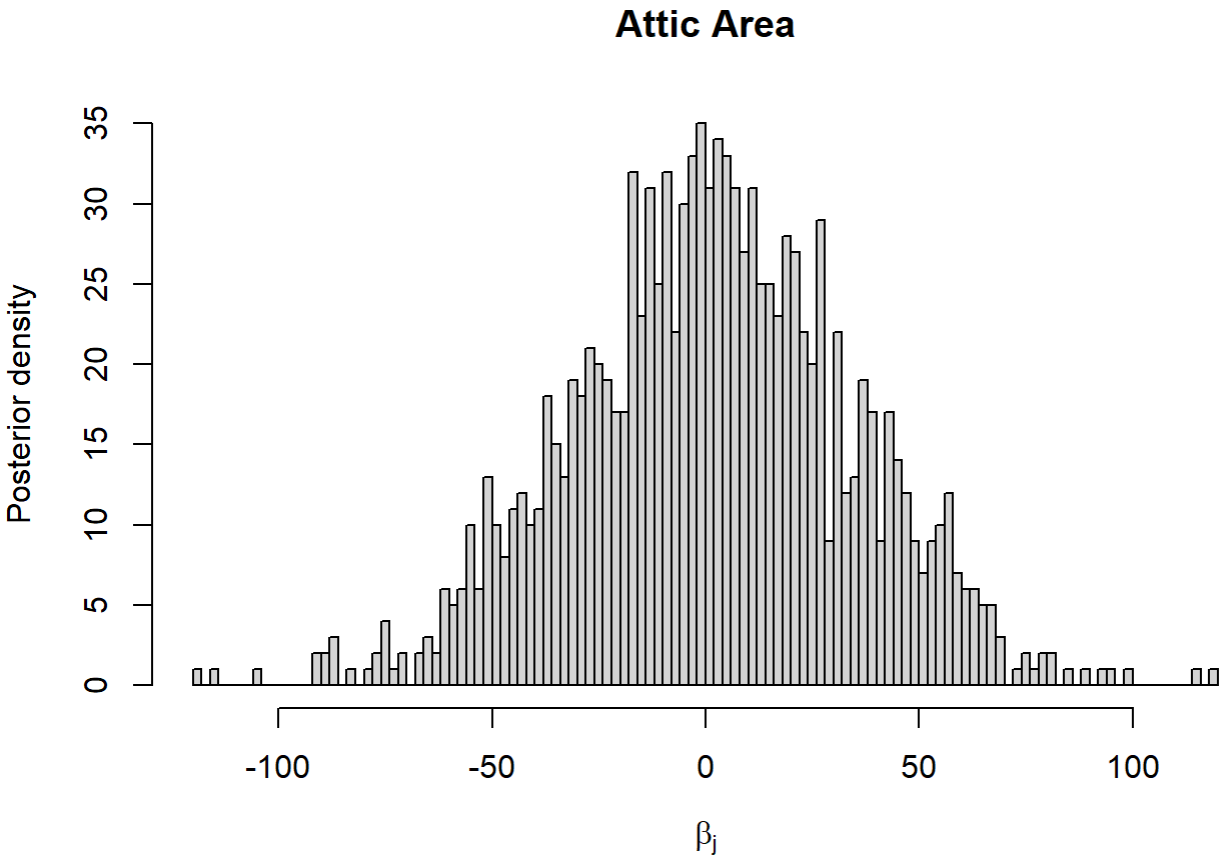


Has Storm Protector

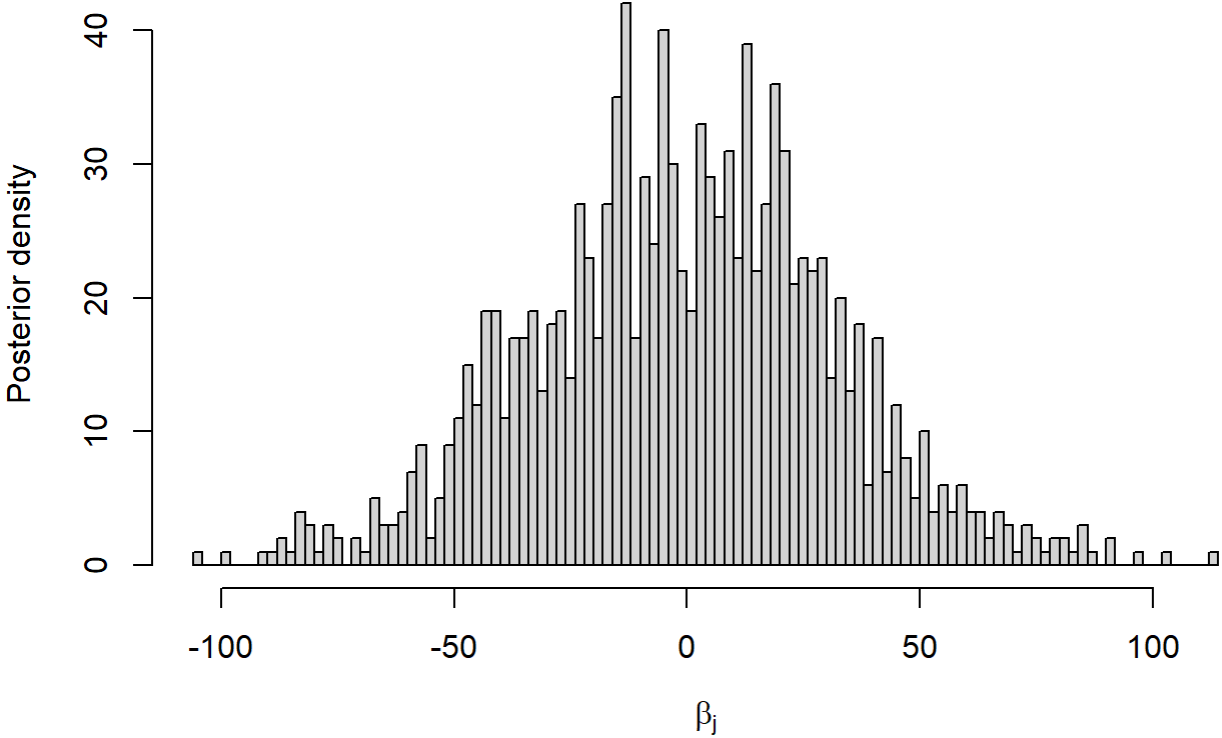


Basement Area

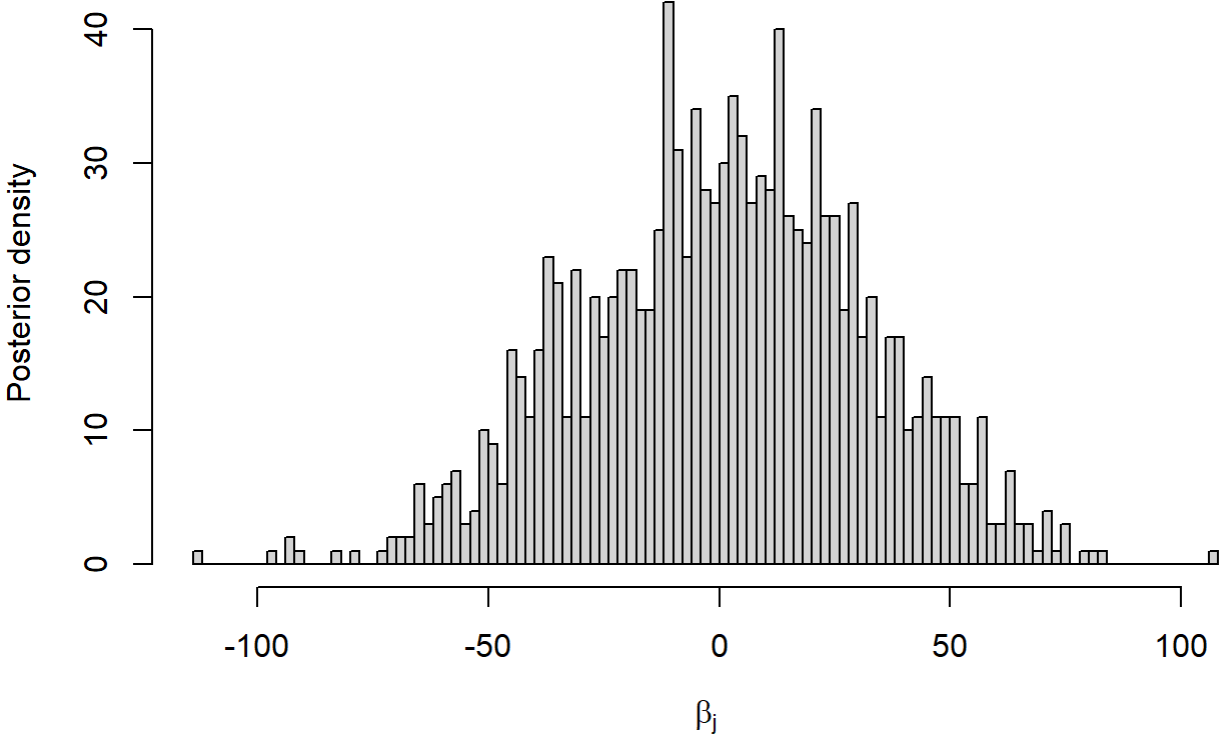




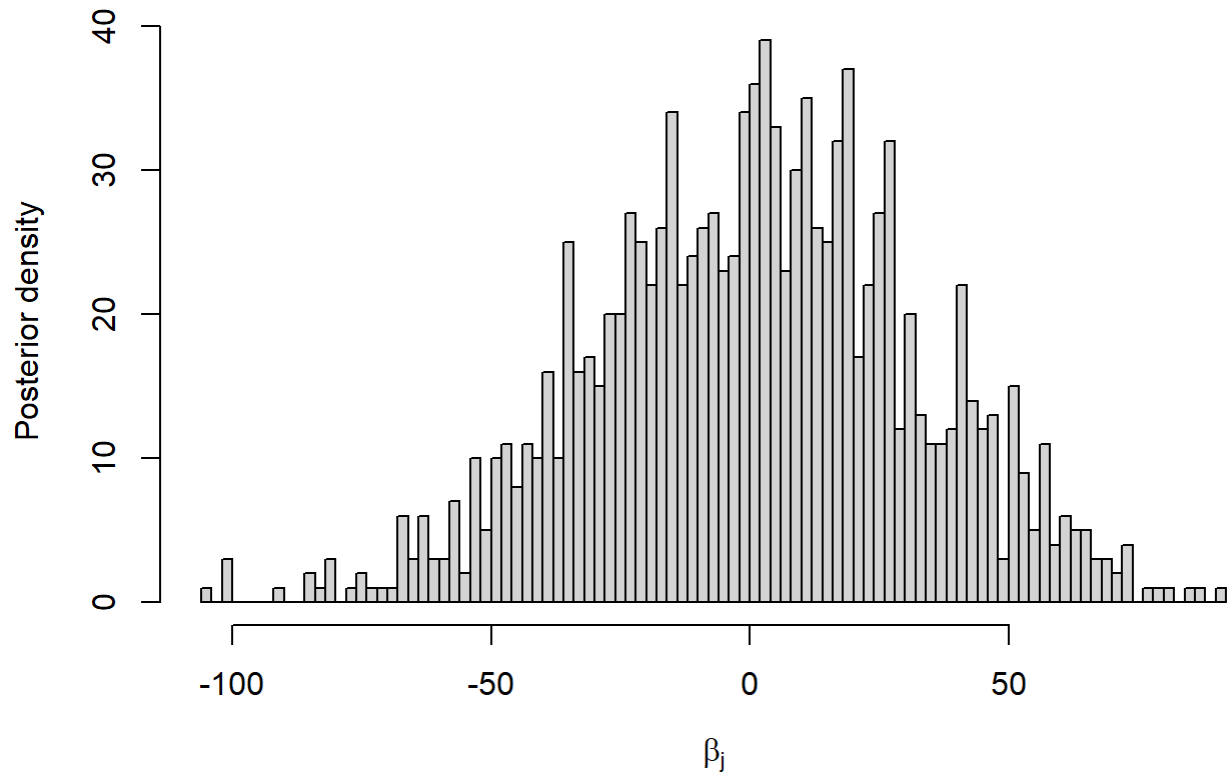
Has Storage Room



Has Guest Room



City Part Range



SSVS

```
library(knitr)
Inc_Prob <- apply(beta!=0,2,mean)
Q <- t(apply(beta,2,quantile,c(0.5,0.05,0.95)))
out <- cbind(Inc_Prob,Q)
kable(round(out,2))
```

	Inc_Prob	50%	5%	95%
price	1	0.32	-51.37	52.35
Square Meters	1	1.67	-47.93	53.97
Number of Rooms	1	-0.96	-51.06	53.22
Has Yard	1	2.46	-50.07	51.40
Has Pool	1	-1.35	-53.11	53.59
Number of Floors	1	-0.81	-54.41	52.76
City Code	1	-0.19	-51.77	51.35
Number of Previous Owners	1	0.59	-53.53	52.70
Year Made	1	-1.29	-52.71	49.91

is Newly Built	1	0.66	-52.15	53.19
Has Storm Protector	1	-0.22	-49.54	53.99
Basement Area	1	0.03	-54.04	52.71
Attic Area	1	1.24	-52.03	55.27
Garage Area	1	-0.62	-48.85	51.82
Has Storage Room	1	-0.84	-52.34	51.45
Has Guest Room	1	2.28	-49.75	51.16
City Part Range	1	1.94	-50.46	52.75

Model 2 (Using features that have strong correlation with the target variable based on the frequentist linear regression model)

Build frequentist linear regression model

```
price.lm <- lm(price ~ X, data = df)

summary(price.lm)
```

```
##
## Call:
## lm(formula = price ~ X, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6988.9 -1192.2   -3.2   1198.7   7005.6
##
## Coefficients:
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept)    4.993e+06  1.898e+01 263030.498  < 2e-16 ***
## XsquareMeters    2.877e+06  1.900e+01 151480.243  < 2e-16 ***
## XnumberOfRooms    7.257e+00  1.901e+01    0.382  0.702640
## XhasYard          1.506e+03  1.899e+01   79.289  < 2e-16 ***
## XhasPool          1.489e+03  1.900e+01   78.365  < 2e-16 ***
## Xfloors           1.576e+03  1.900e+01   82.943  < 2e-16 ***
## XcityCode        -2.332e+01  1.899e+01   -1.228  0.219513
## XnumPrevOwners   -1.130e+00  1.901e+01   -0.059  0.952580
## Xmade            -2.153e+01  1.899e+01   -1.134  0.256940
## XisNewBuilt       7.903e+01  1.900e+01    4.159  3.22e-05 ***
## XhasStormProtector 7.060e+01  1.899e+01    3.718  0.000202 ***
## Xbasement        -6.057e+00  1.900e+01   -0.319  0.749846
## Xattic           -1.305e+01  1.900e+01   -0.687  0.492064
## Xgarage           2.973e+01  1.901e+01    1.563  0.117967
```

```
## XhasStorageRoom      9.743e+00  1.901e+01      0.512 0.608332
## XhasGuestRoom        -1.785e+01  1.901e+01     -0.939 0.347641
## XcityPartRange       1.360e+02  1.900e+01      7.162 8.51e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1898 on 9983 degrees of freedom
## Multiple R-squared:      1, Adjusted R-squared:      1
## F-statistic: 1.436e+09 on 16 and 9983 DF, p-value: < 2.2e-16
```

There are 7 variables that have a strong correlation with the variable price (indicated by 3 stars). These seven variables include squareMeters, hasYard, hasPool, floors, cityPartRange, isNewBuilt, and hasStormProtector.

Split dependent and independent variable

Using 7 variables that have a strong correlation with the variable price

```
X <- df[, c("squareMeters", "hasYard", "hasPool", "floors", "cityPartRange", "isNewBuilt", "hasStormProtector")]
Y <- df$price
names <- c("price", "Square Meters", "Has Yard", "Has Pool", "Floors", "City Part Range", "Is New Built", "Has Storm Protector")
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y, X=X, n=n, p=p)
params <- c("alpha", "beta")
burn <- 500
n.iter <- 2000
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- inprod(X[i,],beta[])
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Set Initial Value

```
inits = list()

inits$alpha = rnorm(1)
for(i in 1:p) {
  inits$beta[i] = rnorm(1)
}
inits$taue = 10
```

Compile Model

```
model2 <- jags.model(model_string, data = data, n.chains=n.chains, quiet=TRUE, inits = inits)
```

Update Model

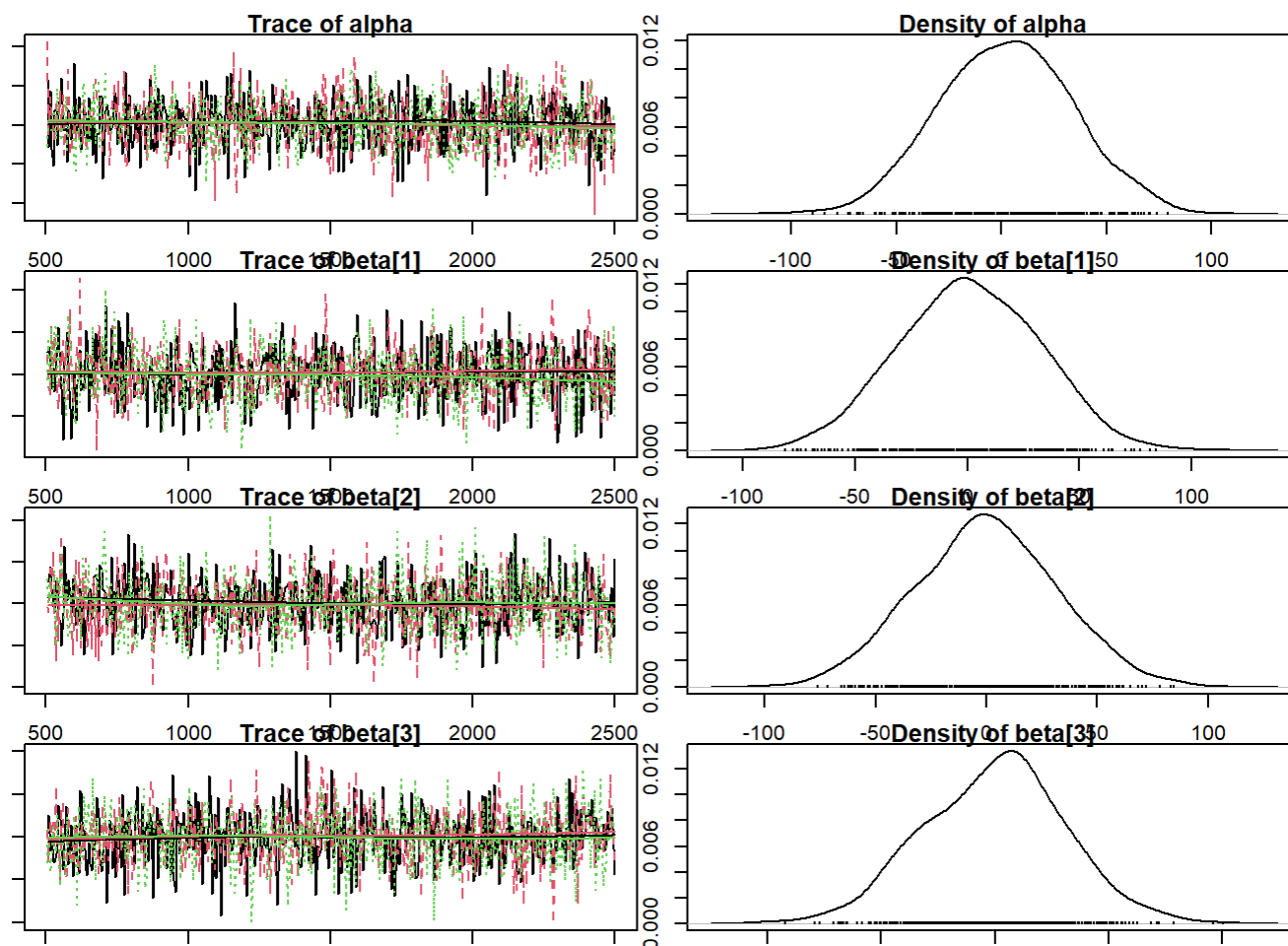
```
update(model2, burn)
```

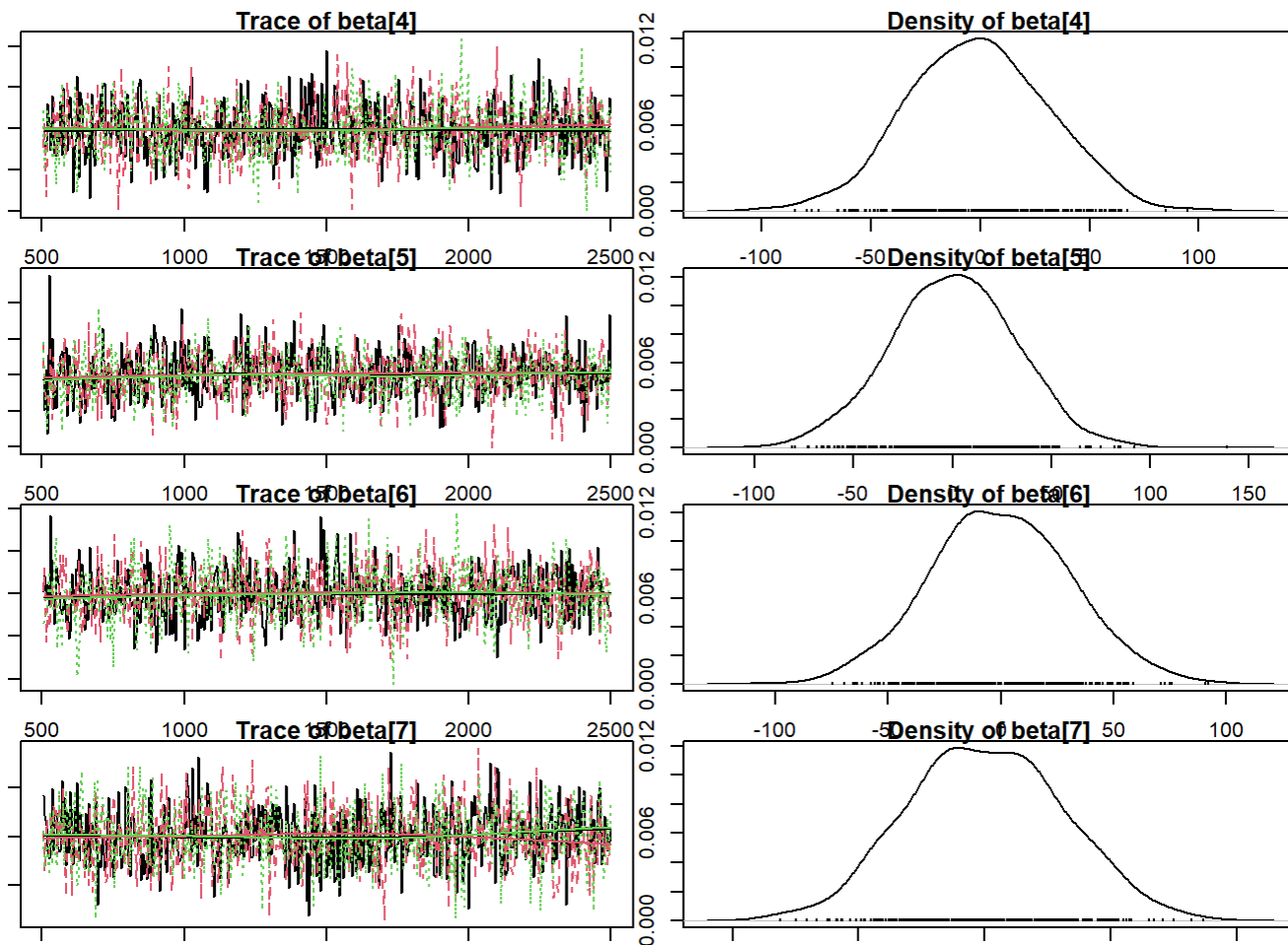
Get Posterior Samples from the model

```
samples2 <- coda.samples(model2, variable.names=params, n.iter=n.iter, thin = thin)
```

Plot MCMC Chain Trace and Features Posterior Density

```
par(mar=c(1,1,1,1))
plot(samples2)
```





Descriptive Statistics of model2

```
summary(samples2)
```

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##      Mean      SD Naive SE Time-series SE
## alpha      2.0525 31.64   0.9133      0.9108
## beta[1]     0.7904 31.31   0.9039      0.8783
## beta[2]    -0.2287 31.82   0.9186      0.9180
## beta[3]    -0.3723 31.00   0.8948      0.8950
## beta[4]    -1.1876 32.13   0.9274      0.9281
## beta[5]     0.3618 31.63   0.9132      0.9405
## beta[6]    -0.4141 31.15   0.8994      0.9341
## beta[7]     0.3345 30.65   0.8849      0.8213
##
## 2. Quantiles for each variable:
```



```
##
##           2.5%    25%      50%   75% 97.5%
## alpha   -57.25 -19.66  2.32298 23.39 64.11
## beta[1] -61.35 -20.70  0.26874 22.49 59.43
## beta[2] -62.19 -21.71 -0.04239 20.95 61.47
## beta[3] -63.79 -21.60  1.91661 19.84 60.22
## beta[4] -64.93 -23.16 -1.18507 20.49 60.09
## beta[5] -62.16 -20.58  0.92715 20.81 60.61
## beta[6] -61.50 -21.32 -0.42023 20.89 60.98
## beta[7] -57.72 -20.29  0.04519 21.37 58.86
```

Show Alpha and Betas to the corresponding Features

```
sum <- summary(samples2)
rownames(sum$statistics) <- names
rownames(sum$quantiles) <- names
sum$statistics <- round(sum$statistics,3)
sum$quantiles <- round(sum$quantiles,3)
sum
```

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## price           2.053 31.64    0.913      0.911
## Square Meters    0.790 31.31    0.904      0.878
## Has Yard        -0.229 31.82    0.919      0.918
## Has Pool        -0.372 31.00    0.895      0.895
## Floors          -1.188 32.13    0.927      0.928
## City Part Range  0.362 31.63    0.913      0.940
## Is New Built    -0.414 31.16    0.899      0.934
## Has Storm Protector 0.335 30.65    0.885      0.821
##
## 2. Quantiles for each variable:
##
##           2.5%    25%      50%   75% 97.5%
## price      -57.25 -19.66  2.323 23.39 64.11
## Square Meters -61.35 -20.70  0.269 22.49 59.43
## Has Yard     -62.19 -21.71 -0.042 20.95 61.47
## Has Pool     -63.79 -21.60  1.917 19.84 60.22
## Floors       -64.93 -23.16 -1.185 20.48 60.09
## City Part Range -62.16 -20.58  0.927 20.81 60.61
## Is New Built  -61.50 -21.32 -0.420 20.89 60.98
```

```
## Has Storm Protector -57.72 -20.29 0.045 21.37 58.86
```

Check Convergence

```
gelman.diag(samples2)
```

```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## alpha           1.004         1.01
## beta[1]          1.005         1.02
## beta[2]          1.002         1.01
## beta[3]          1.001         1.01
## beta[4]          0.999         1.00
## beta[5]          1.002         1.01
## beta[6]          1.000         1.00
## beta[7]          0.999         1.00
##
## Multivariate psrf
##
## 1.01
```

Compile results

```
ESS2 <- effectiveSize(samples2)
out2 <- summary(samples2)$quantiles
rownames(out2)<-names
```

```
ESS2
```

```
##      alpha  beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
## 1264.295 1283.819 1200.000 1200.000 1200.000 1141.339 1119.811 1424.617
```

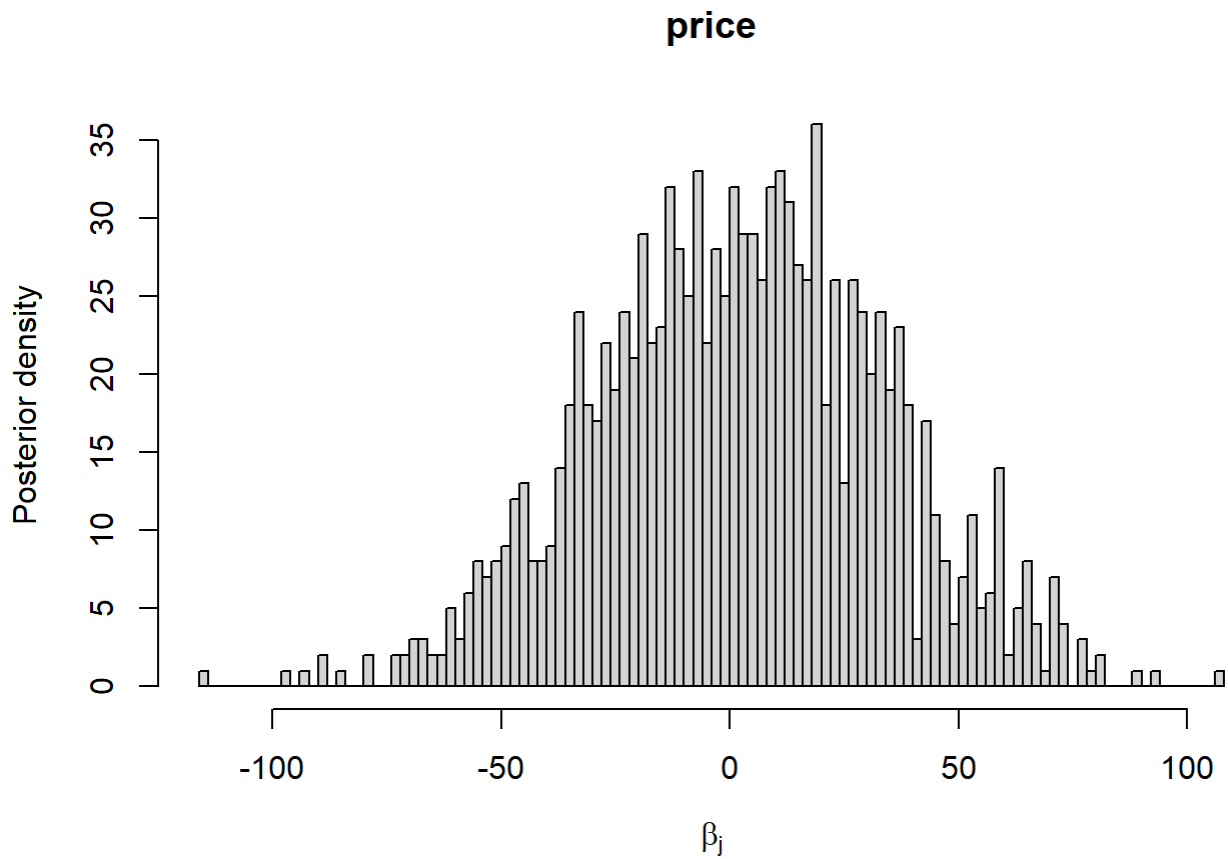
Compute DIC & WAIC

```
# DIC
dic2 <- dic.samples(model2,n.iter=n.iter)

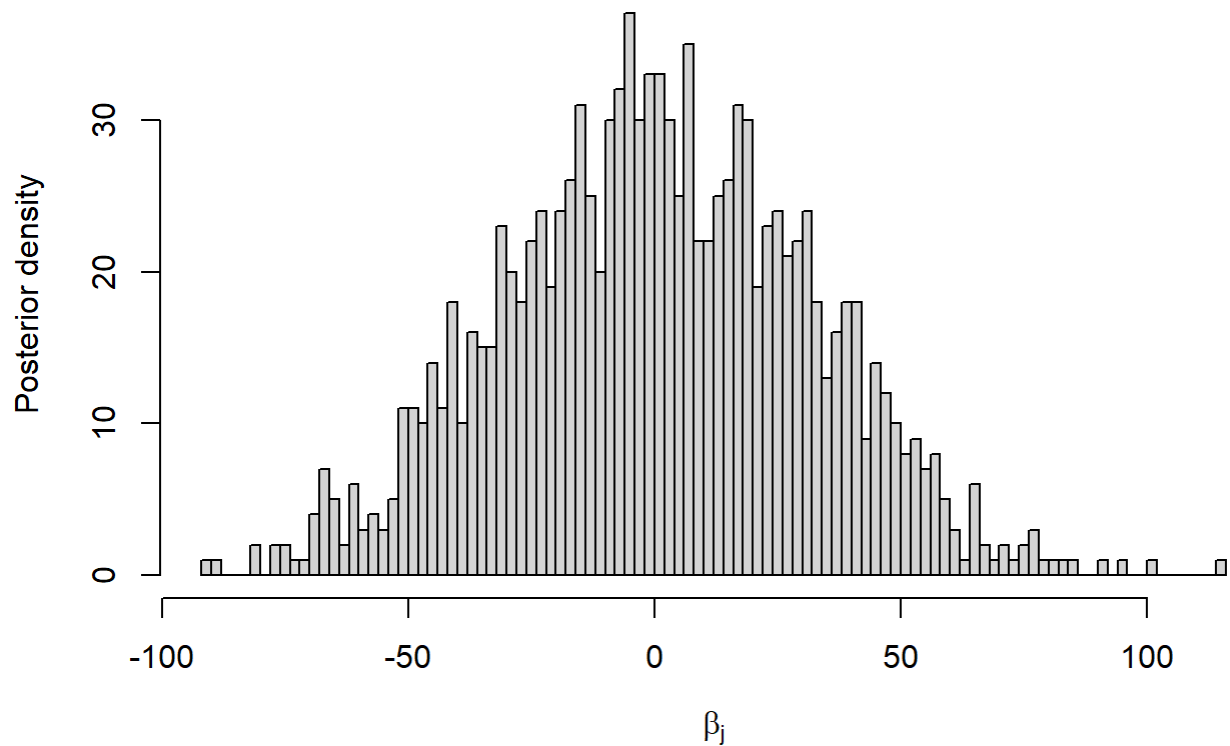
# WAIC
waic2 <- coda.samples(model2, variable.names=c("like"), n.iter=n.iter)
like2 <- waic2[[1]]
fbar2 <- colMeans(like2)
P2 <- sum(apply(log(like2),2,var))
WAIC2 <- -2*sum(log(fbar2))+2*P2
```

Plot Posterior Density Individually

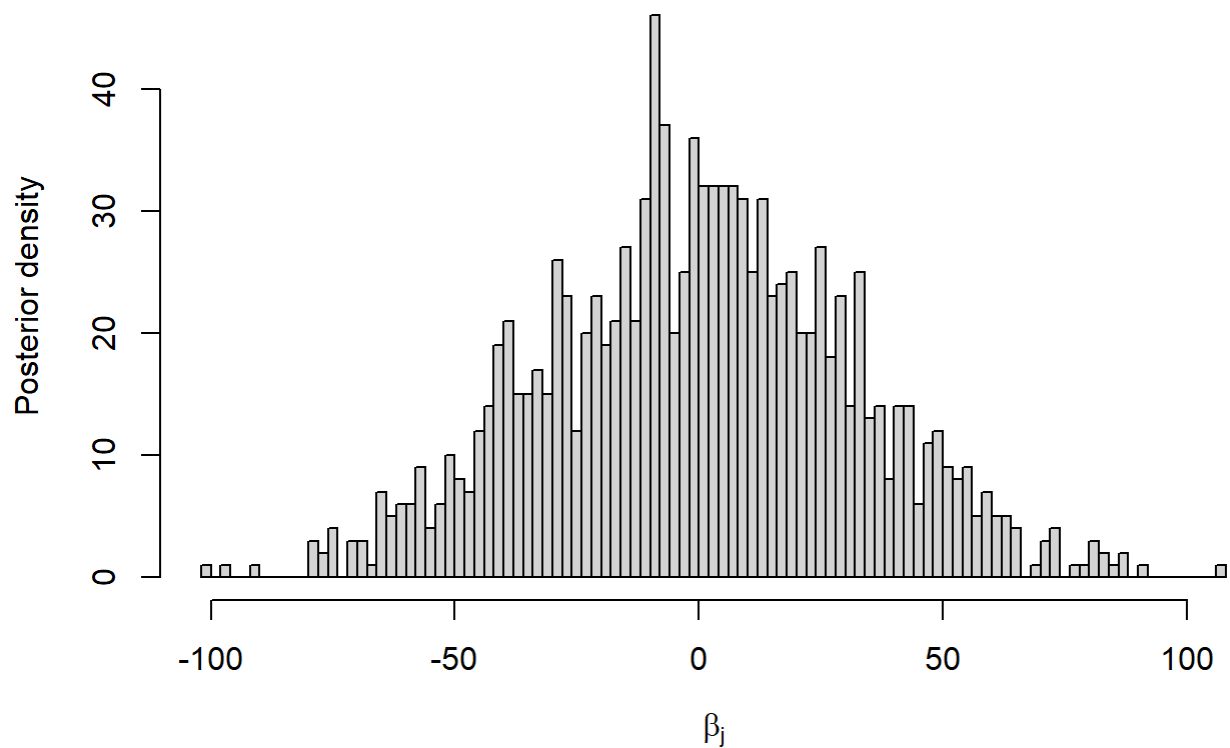
```
beta <- NULL
for(l in 1:n.chains){
  beta <- rbind(beta,samples2[[l]])
}
colnames(beta) <- names
for(j in 1:8){
  hist(beta[,j],xlab=expression(beta[j]),ylab="Posterior density",
  breaks=100,main=names[j])
}
```

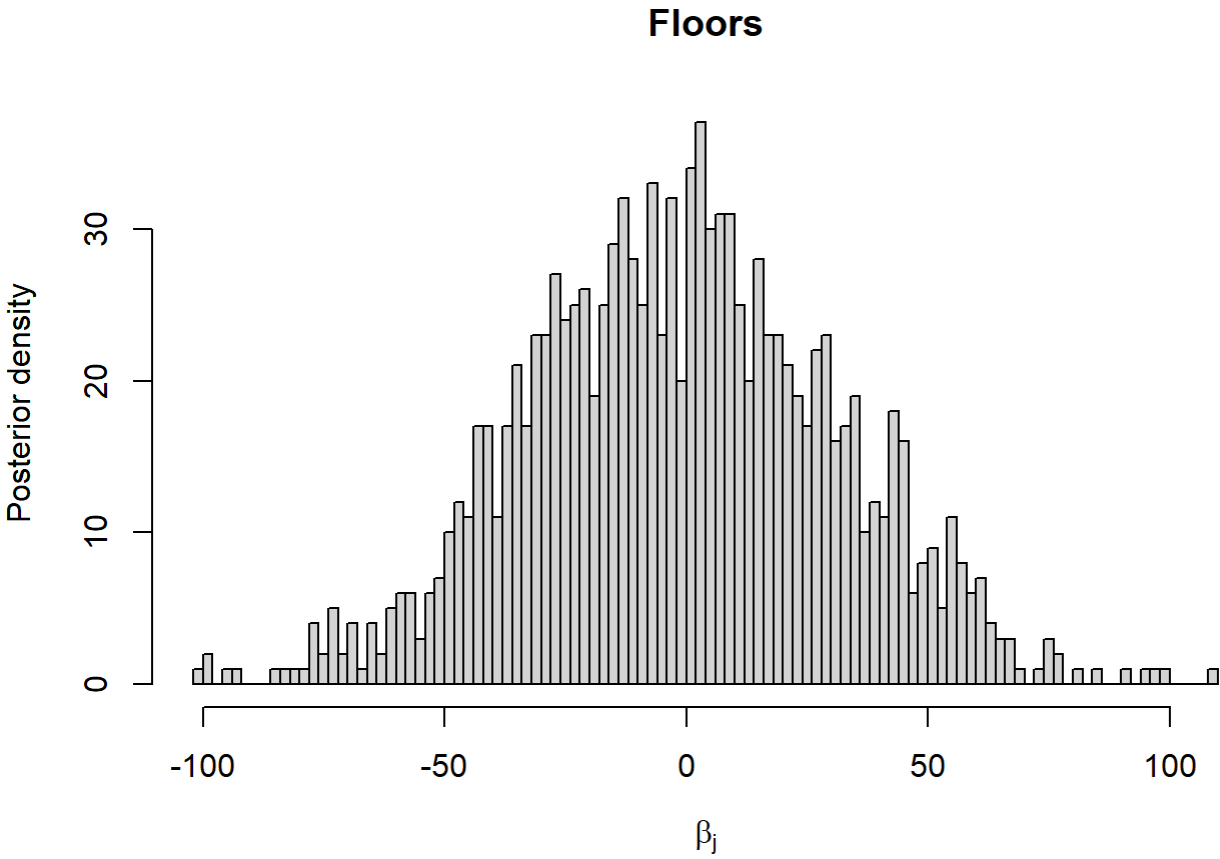
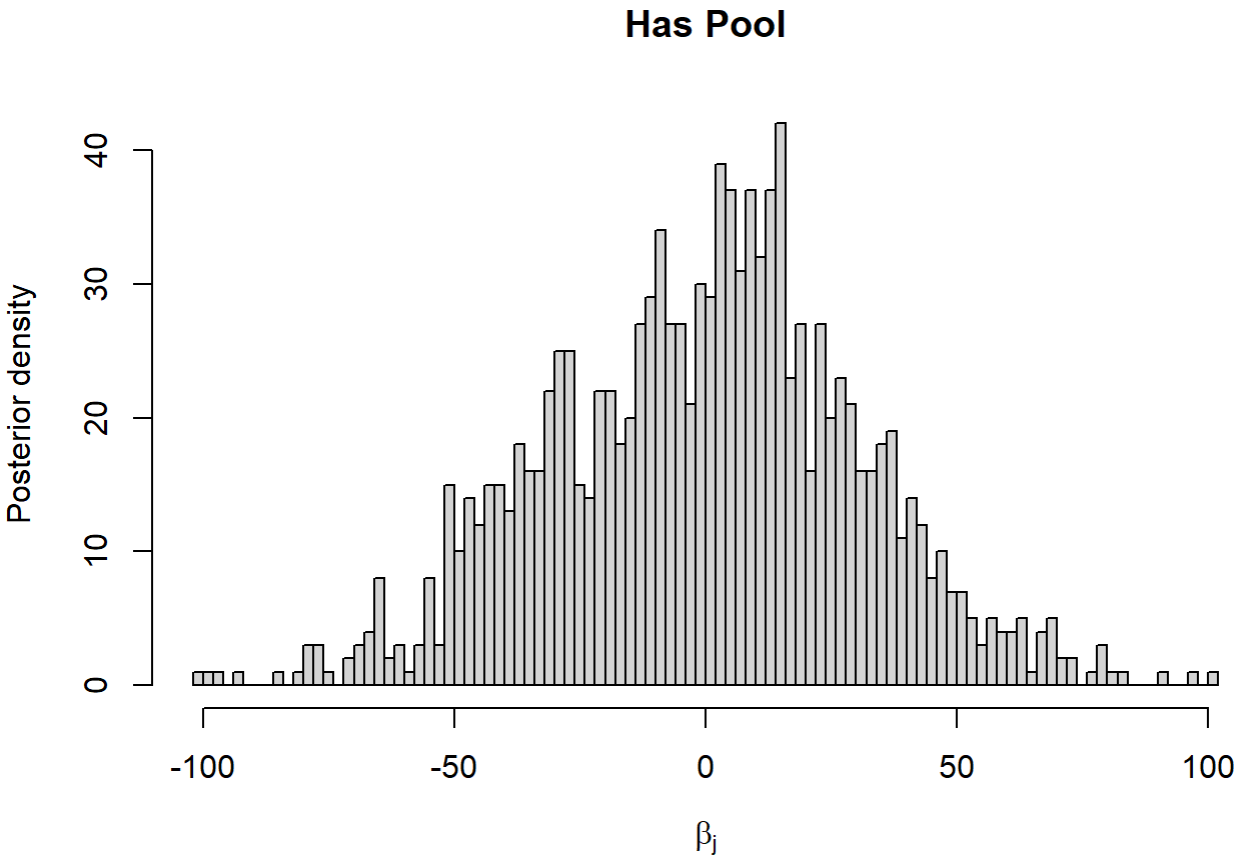


Square Meters

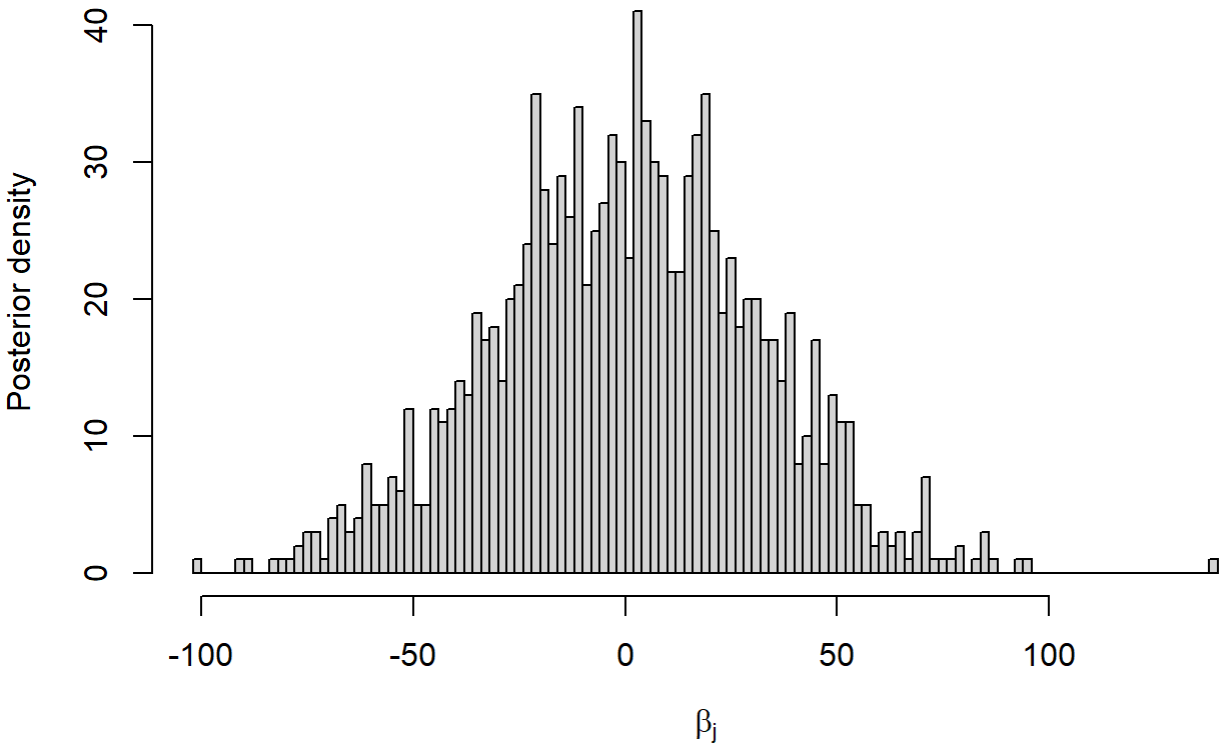


Has Yard

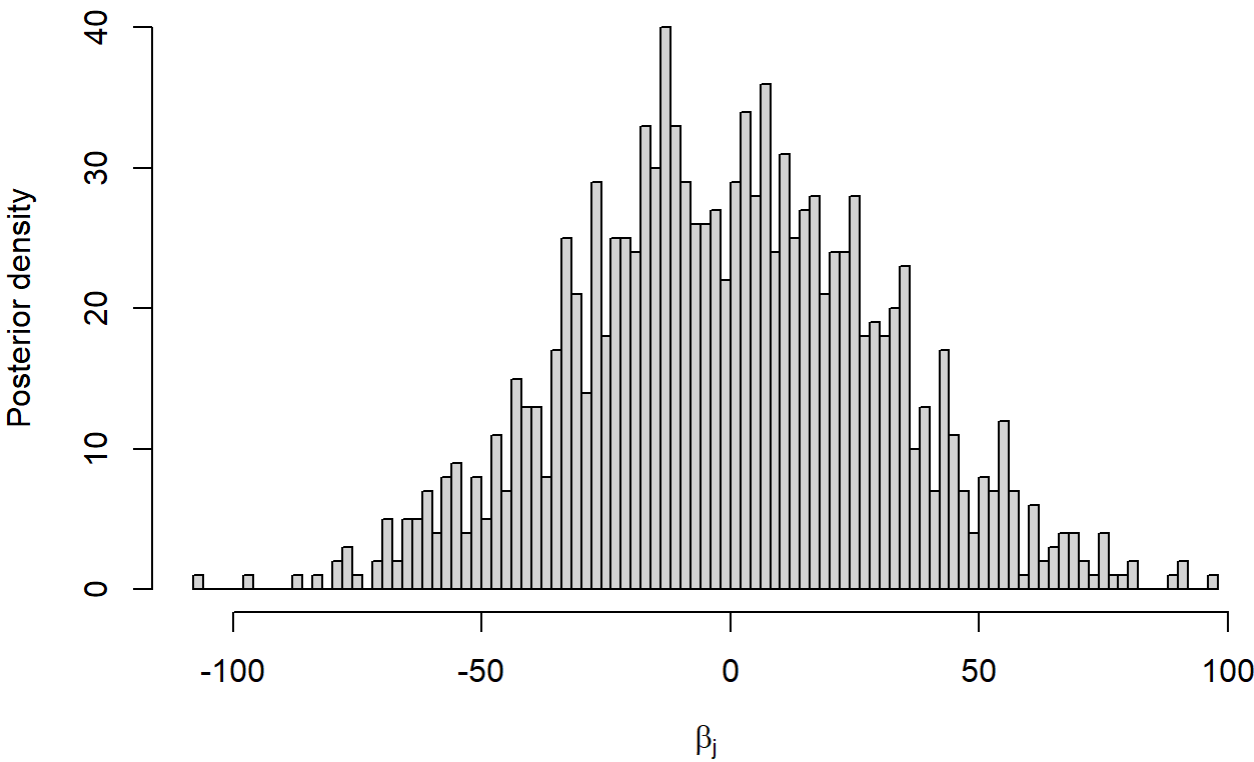


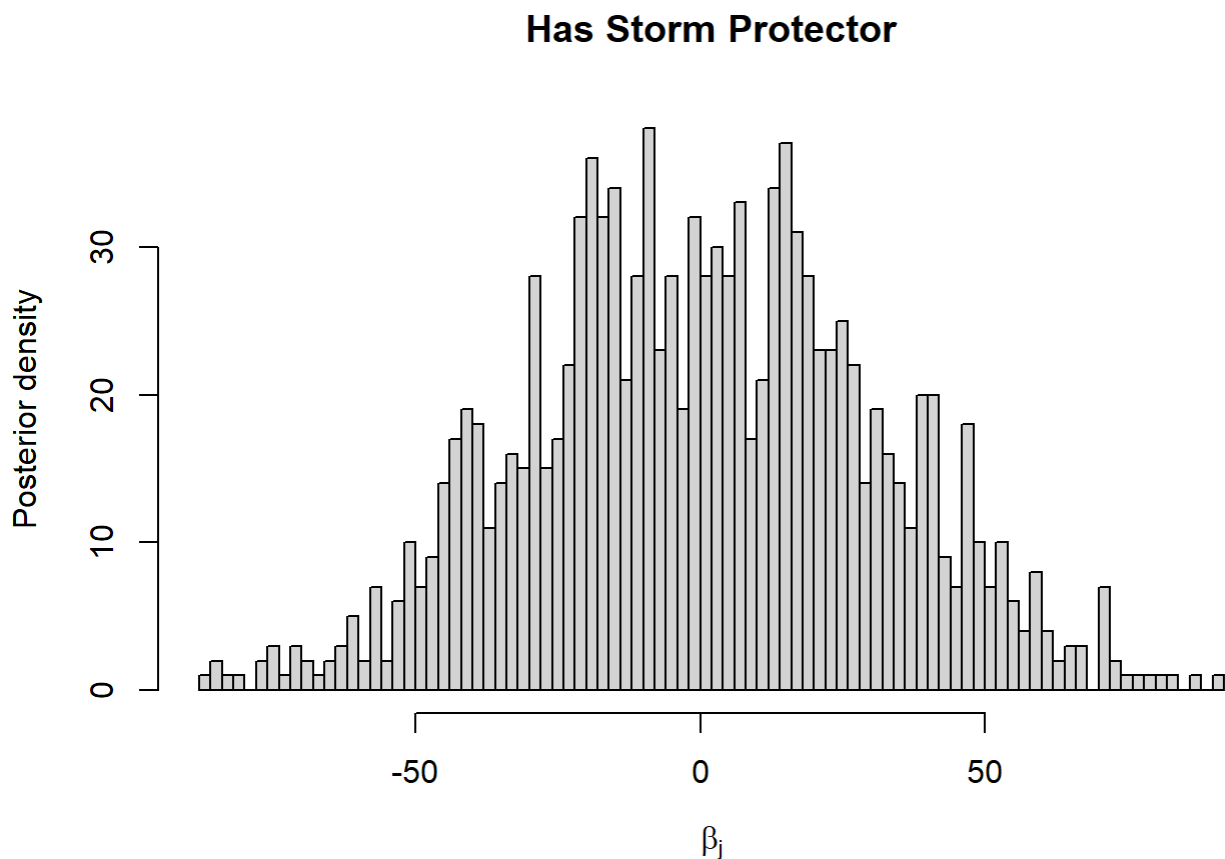


City Part Range



Is New Built





Model 3 (Same like model 2, but using initial values (alpha and beta coefficients) from the estimation of frequentist linear model)

Split dependent and independent variable

```
X <- df[, c("squareMeters", "hasYard", "hasPool", "floors", "cityPartRange", "isNewBuilt", "hasStormProtector")]
Y <- df$price
names <- c("price", "Square Meters", "Has Yard", "Has Pool", "Floors", "City Part Range", "Is New Built", "Has Storm Protector")
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y,X=X,n=n,p=p)
params <- c("alpha","beta")
burn <- 500
n.iter <- 2000
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- inprod(X[i,],beta[])
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Frequentist Linear Regression

```
price.lm2 <- lm(price ~ X, data = df)

summary(price.lm2)
```



```
##
## Call:
## lm(formula = price ~ X, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6976.4  -1193.3    -5.9   1201.2   6968.3
##
## Coefficients:
##              Estimate Std. Error  t value Pr(>|t|)
## (Intercept)   4.993e+06  1.898e+01  2.631e+05 < 2e-16 ***
## XsquareMeters  2.877e+06  1.899e+01  1.516e+05 < 2e-16 ***
## XhasYard       1.506e+03  1.899e+01  7.930e+01 < 2e-16 ***
## XhasPool       1.489e+03  1.899e+01  7.841e+01 < 2e-16 ***
## Xfloors        1.576e+03  1.898e+01  8.303e+01 < 2e-16 ***
## XcityPartRange 1.355e+02  1.899e+01  7.135e+00 1.04e-12 ***
## XisNewBuilt    7.868e+01  1.899e+01  4.144e+00 3.43e-05 ***
## XhasStormProtector 7.105e+01  1.899e+01  3.742e+00 0.000183 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1898 on 9992 degrees of freedom
## Multiple R-squared:  1, Adjusted R-squared:  1
## F-statistic: 3.282e+09 on 7 and 9992 DF, p-value: < 2.2e-16
```

Linear regression model : $Y_i = 4993000 + 2877000 * \text{squareMeters} + 1506 * \text{hasYard} + 1489 * \text{hasPool} + 1576 * \text{floors} + 135.5 * \text{cityPartRange} + 78.68 * \text{isNewBuilt} + 71.05 * \text{hasStormProtector}$

Set Initial Value

We use the estimation alpha and betas from Frequentist Linear Regression Model

```
inits = list()

inits$alpha = 4993000
inits$beta[1] = 2877000
inits$beta[2] = 1506
inits$beta[3] = 1489
inits$beta[4] = 1576
inits$beta[5] = 135.5
inits$beta[6] = 78.68
inits$beta[7] = 71.05
inits$taue = 10
```

Compile Model

```
model3 <- jags.model(model_string, data = data, n.chains=n.chains, quiet=TRUE, inits = inits)
```

Update Model

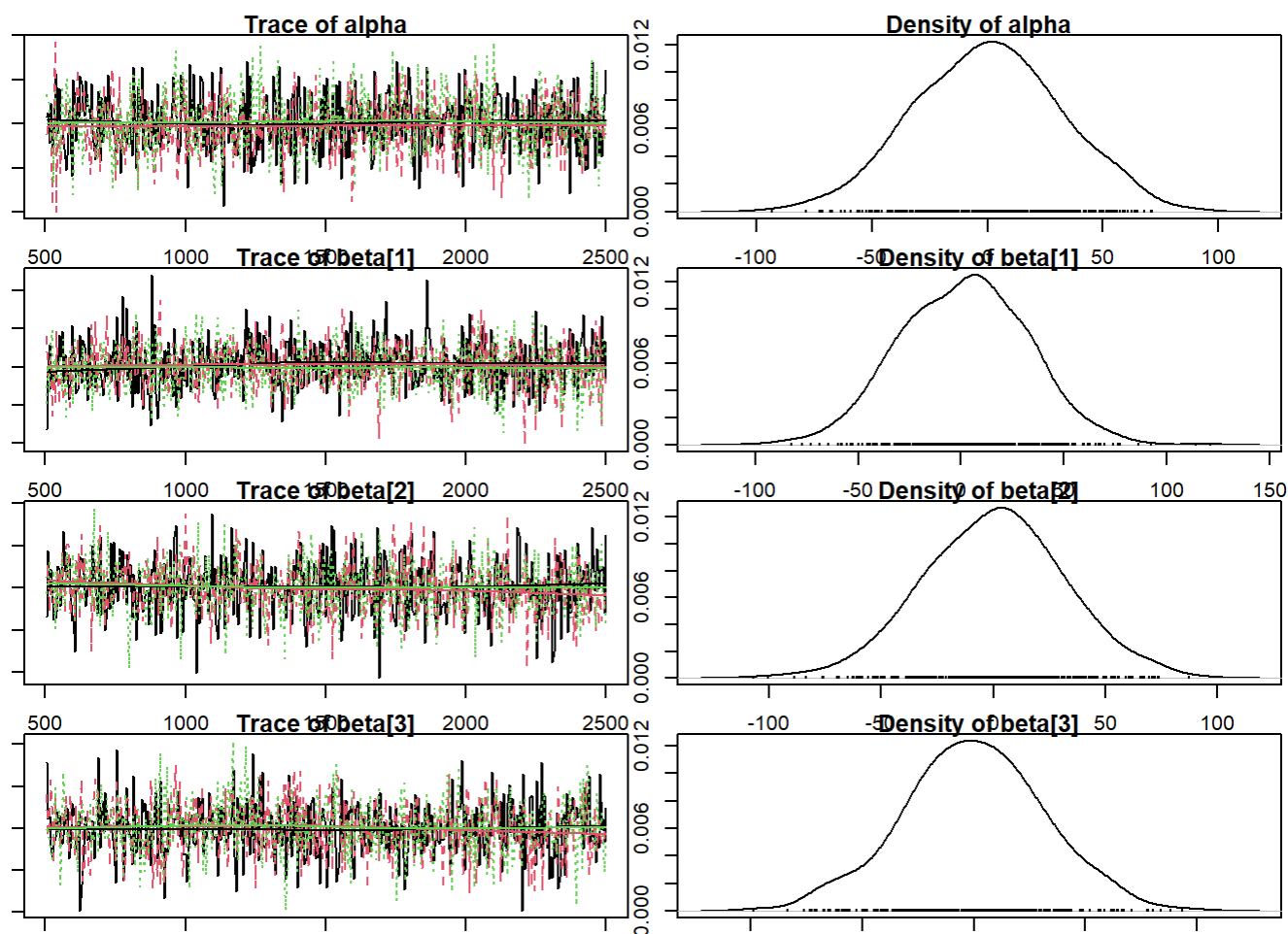
```
update(model3, burn)
```

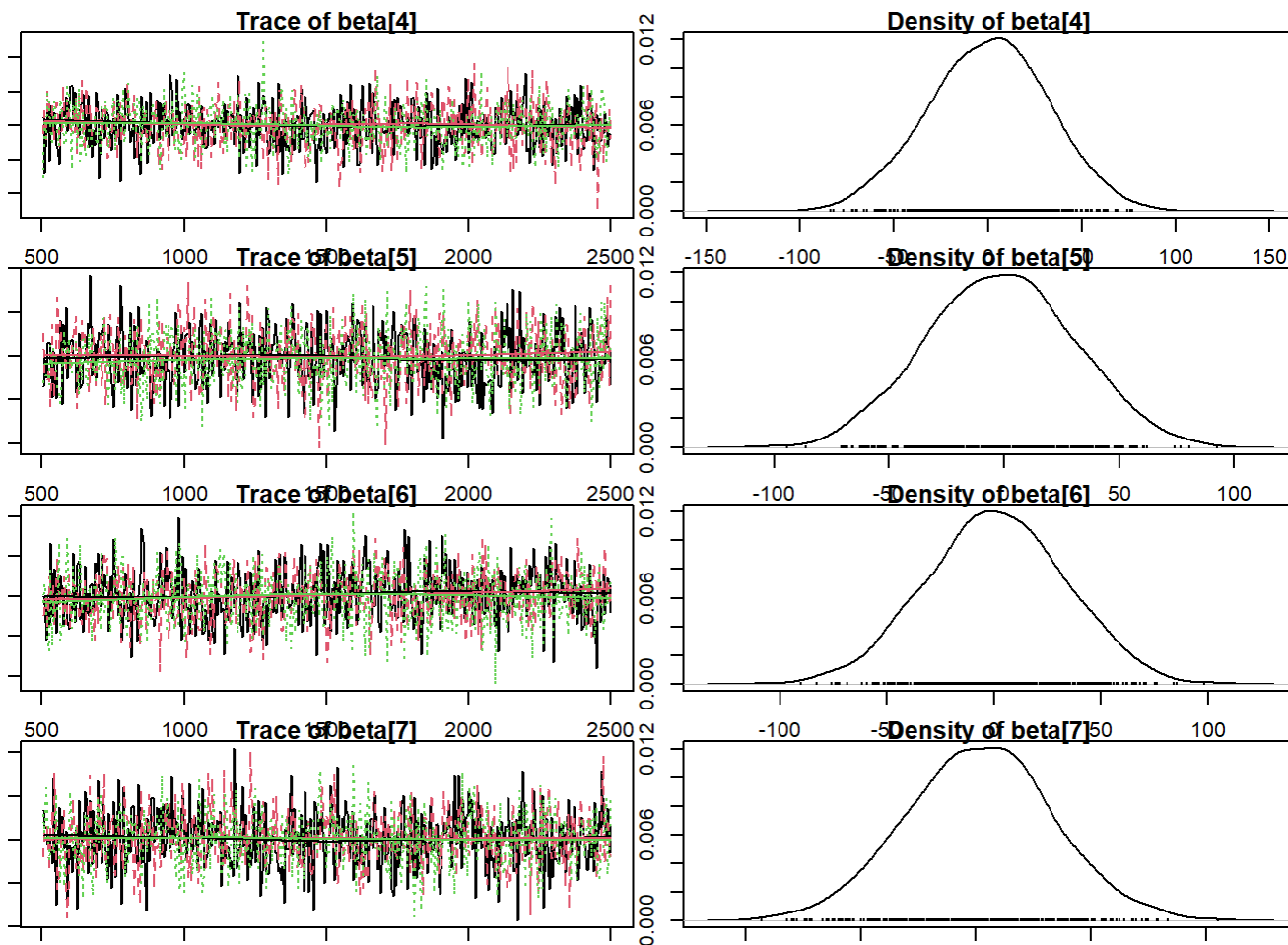
Get Posterior Samples from the model

```
samples3 <- coda.samples(model3, variable.names=params, n.iter=n.iter, thin=thin)
```

Plot MCMC Chain Trace and Features Posterior Density

```
par(mar=c(1,1,1,1))
plot(samples3)
```





Descriptive Statistics of model 3

```
summary(samples3)
```

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##      Mean      SD Naive SE Time-series SE
## alpha      0.2873 31.99  0.9234      0.9950
## beta[1]     1.2720 30.95  0.8936      0.8920
## beta[2]     0.2201 31.46  0.9082      0.9105
## beta[3]    -0.2151 31.95  0.9222      0.9225
## beta[4]     0.2823 31.84  0.9193      0.9200
## beta[5]    -1.6672 31.76  0.9168      0.9478
## beta[6]     0.6613 32.25  0.9309      0.8737
## beta[7]     1.4804 31.66  0.9139      0.9196
##
## 2. Quantiles for each variable:
```

```
##
##           2.5%    25%    50%    75% 97.5%
## alpha   -63.48 -22.09  0.4442 22.06 61.60
## beta[1] -58.61 -20.15  2.1016 22.61 60.98
## beta[2] -62.17 -20.83  0.8050 21.04 63.15
## beta[3] -65.65 -20.95 -0.2200 20.78 61.65
## beta[4] -63.12 -20.91  1.3112 22.21 60.62
## beta[5] -61.45 -23.31 -1.7339 19.21 60.85
## beta[6] -62.84 -20.25  1.0262 22.57 62.42
## beta[7] -59.99 -19.41  1.7640 22.70 62.62
```

Show Alpha and Betas to the corresponding Features

```
sum <- summary(samples3)
rownames(sum$statistics) <- names
rownames(sum$quantiles) <- names
sum$statistics <- round(sum$statistics,3)
sum$quantiles <- round(sum$quantiles,3)
sum
```

```
##
## Iterations = 505:2500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 400
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## price           0.287 31.99    0.923      0.995
## Square Meters    1.272 30.95    0.894      0.892
## Has Yard         0.220 31.46    0.908      0.911
## Has Pool        -0.215 31.95    0.922      0.922
## Floors          0.282 31.84    0.919      0.920
## City Part Range -1.667 31.76    0.917      0.948
## Is New Built     0.661 32.25    0.931      0.874
## Has Storm Protector 1.480 31.66    0.914      0.920
##
## 2. Quantiles for each variable:
##
##           2.5%    25%    50%    75% 97.5%
## price      -63.48 -22.09  0.444 22.06 61.60
## Square Meters -58.61 -20.15  2.102 22.61 60.98
## Has Yard     -62.17 -20.83  0.805 21.04 63.15
## Has Pool     -65.65 -20.95 -0.220 20.78 61.65
## Floors       -63.12 -20.91  1.311 22.21 60.62
## City Part Range -61.45 -23.31 -1.734 19.21 60.85
## Is New Built  -62.84 -20.25  1.026 22.57 62.42
```

```
## Has Storm Protector -59.99 -19.41 1.764 22.70 62.62
```

Check Convergence

```
gelman.diag(samples3)
```

```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## alpha           1.006      1.020
## beta[1]          1.009      1.039
## beta[2]          1.001      1.009
## beta[3]          0.998      0.999
## beta[4]          1.005      1.006
## beta[5]          0.999      1.003
## beta[6]          1.003      1.012
## beta[7]          1.001      1.003
##
## Multivariate psrf
##
## 1.01
```

Compile results

```
ESS3 <- effectiveSize(samples3)
out3 <- summary(samples3)$quantiles
rownames(out3)<-names
```

```
ESS3
```

```
##      alpha  beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
## 1072.648 1200.000 1220.999 1200.000 1200.000 1131.748 1406.552 1188.187
```

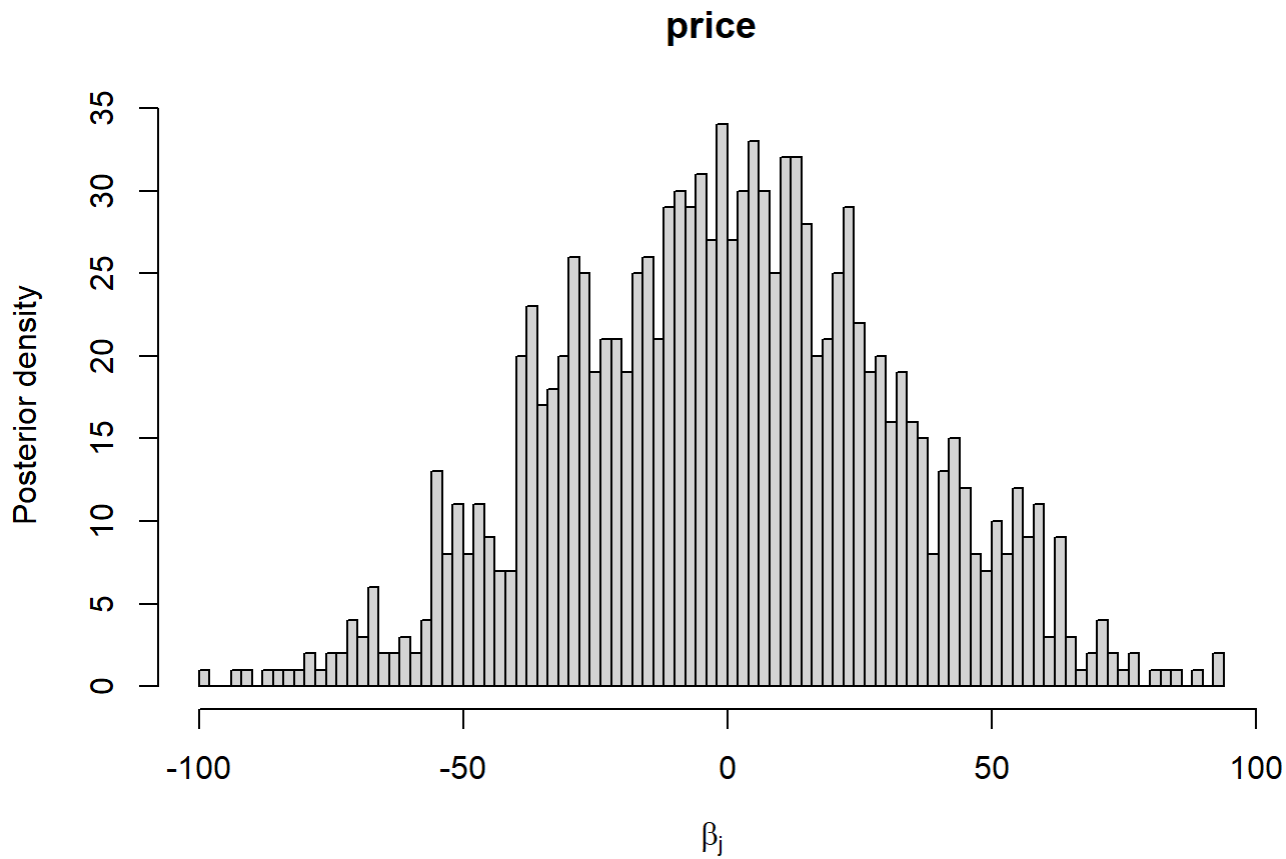
Compute DIC & WAIC

```
# DIC
dic3 <- dic.samples(model3,n.iter=n.iter)

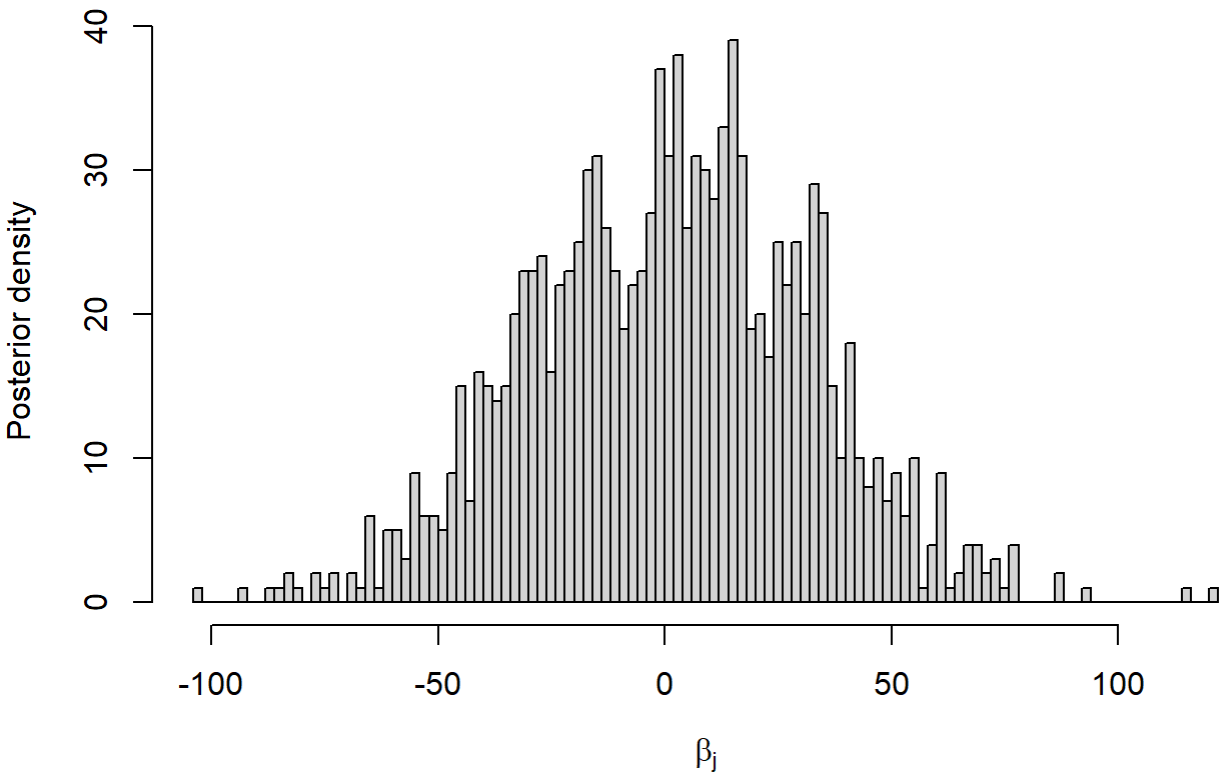
# WAIC
waic3 <- coda.samples(model3, variable.names=c("like"), n.iter=n.iter)
like3 <- waic3[[1]]
fbar3 <- colMeans(like3)
P3 <- sum(apply(log(like3),2,var))
WAIC3 <- -2*sum(log(fbar3))+2*P3
```

Plot Posterior Density Individually

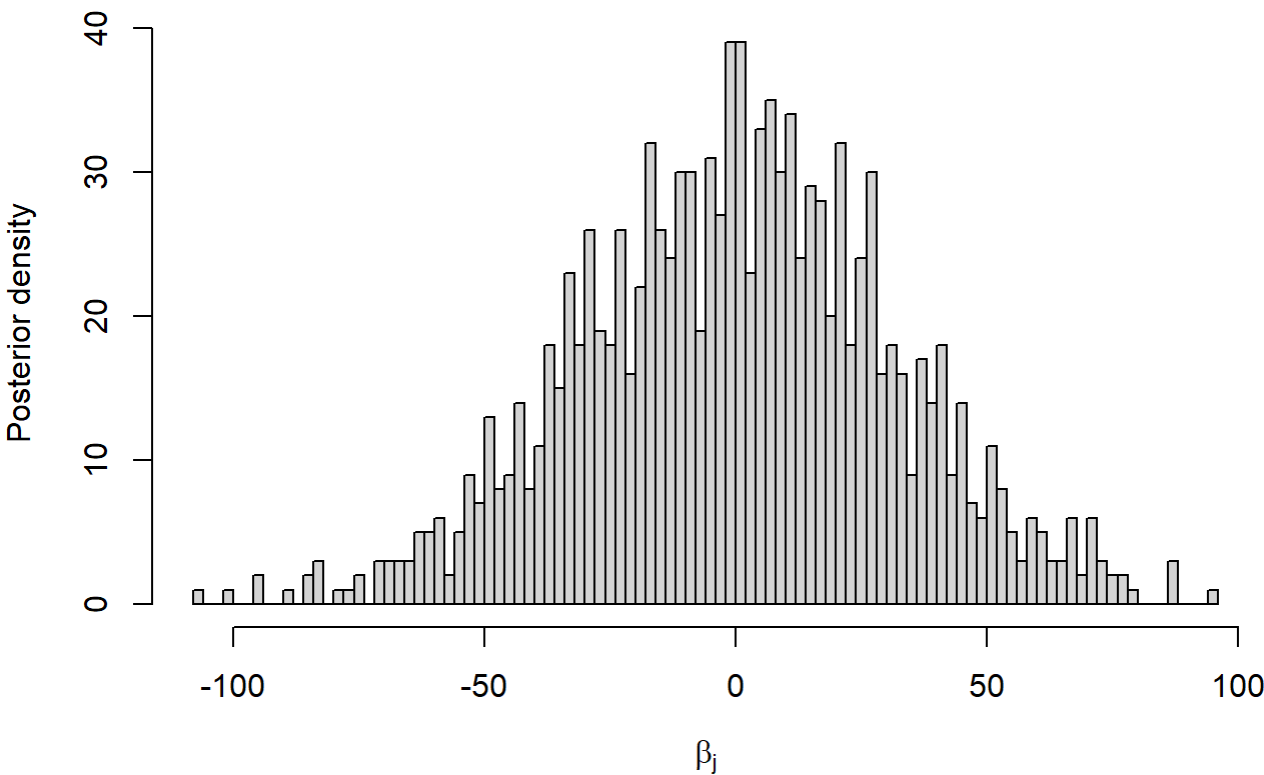
```
beta <- NULL
for(l in 1:n.chains){
  beta <- rbind(beta,samples3[[l]])
}
colnames(beta) <- names
for(j in 1:8){
  hist(beta[,j],xlab=expression(beta[j]),ylab="Posterior density",
    breaks=100,main=names[j])
}
```

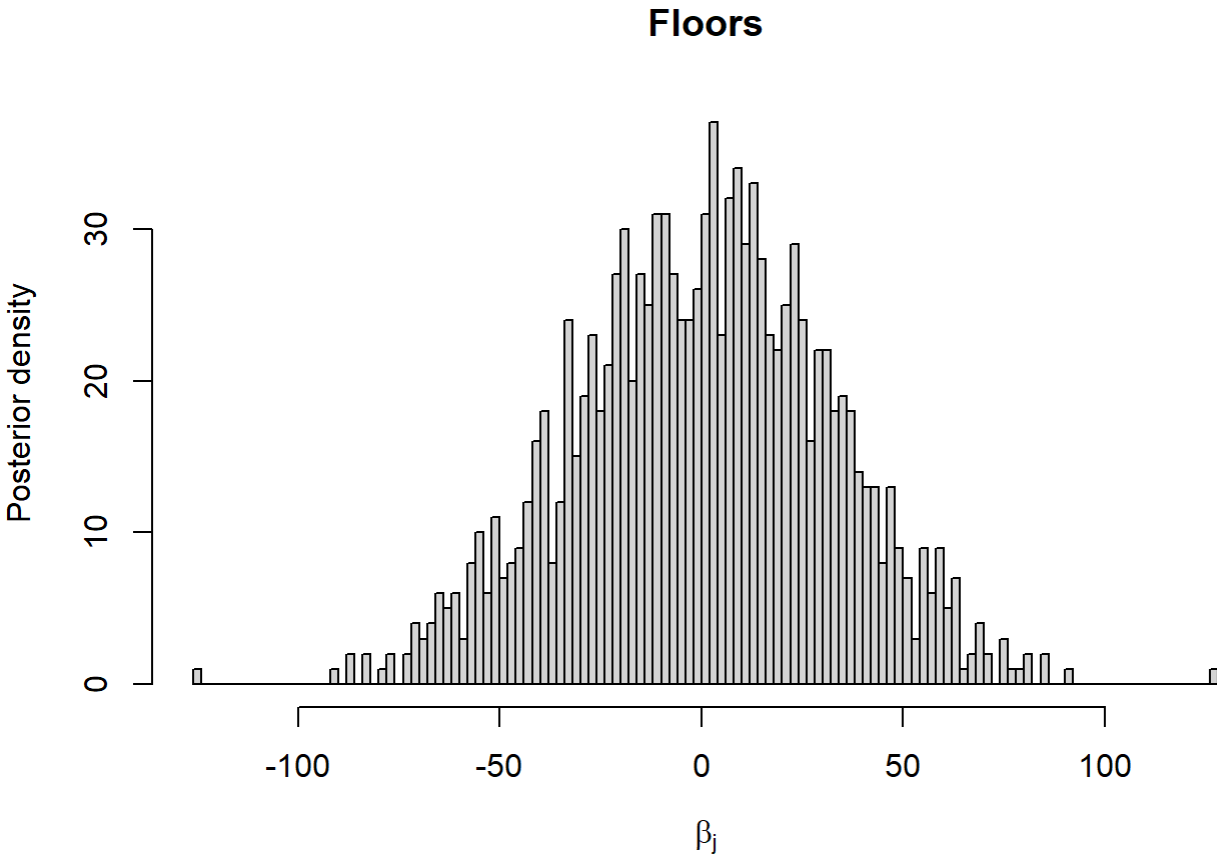
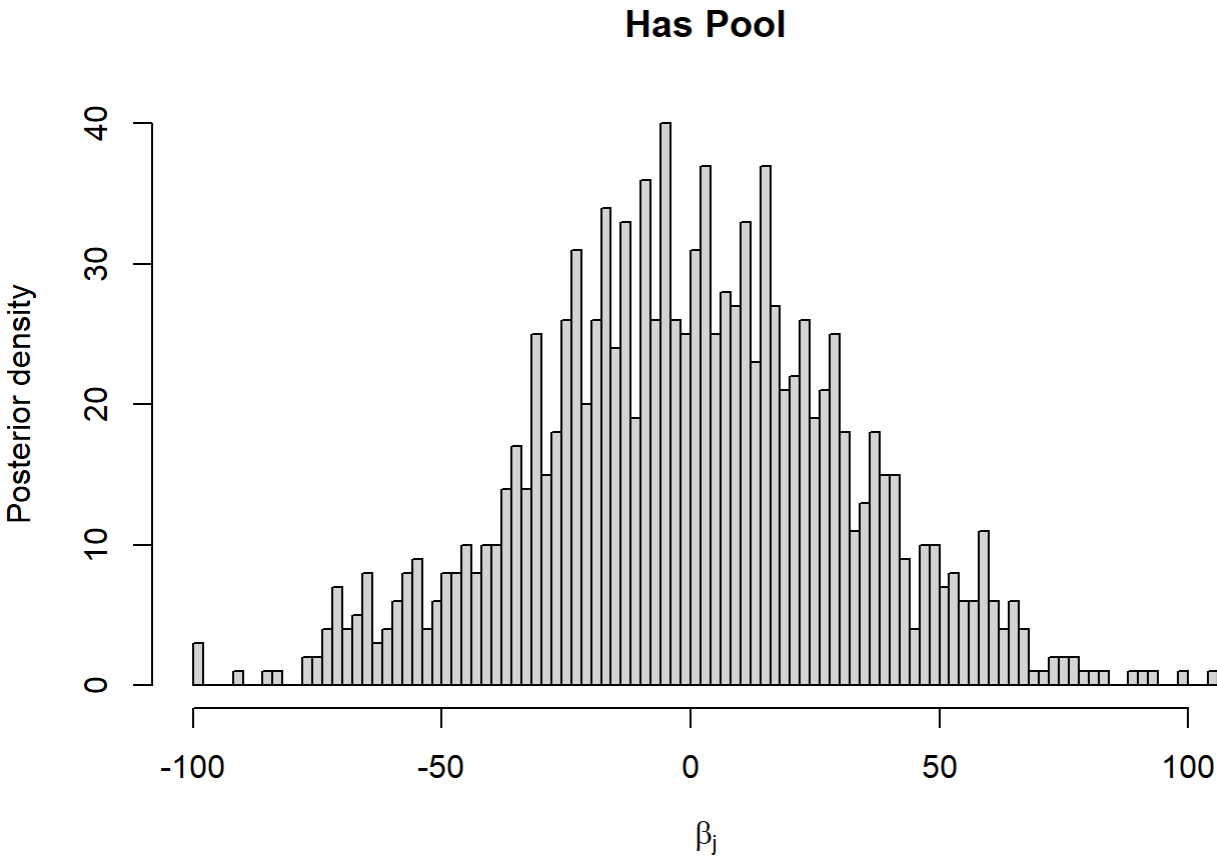


Square Meters

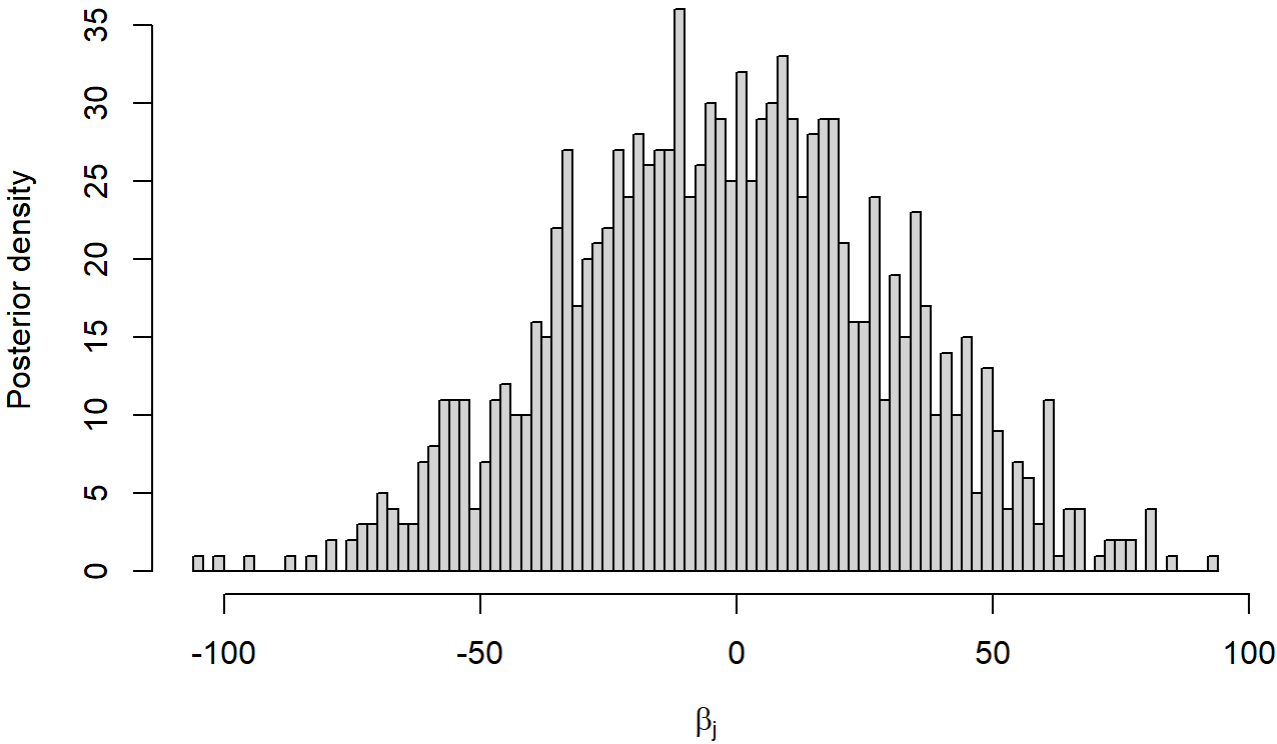


Has Yard

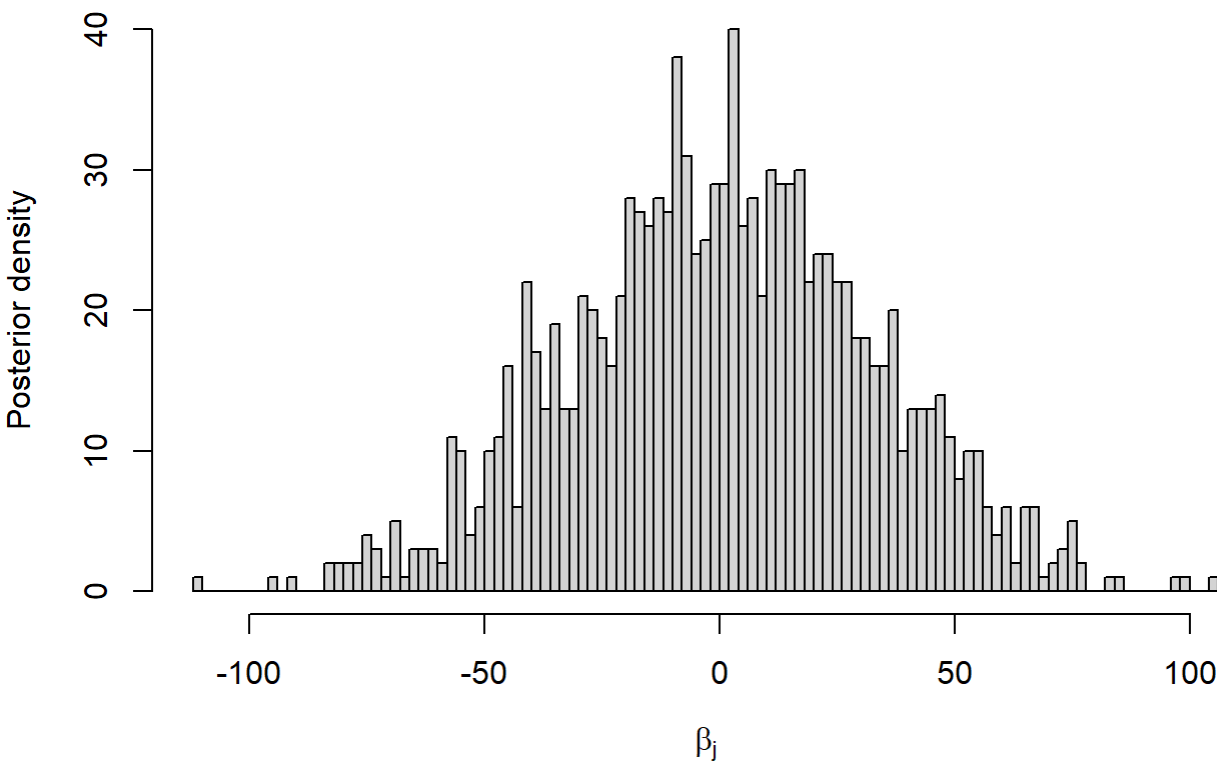




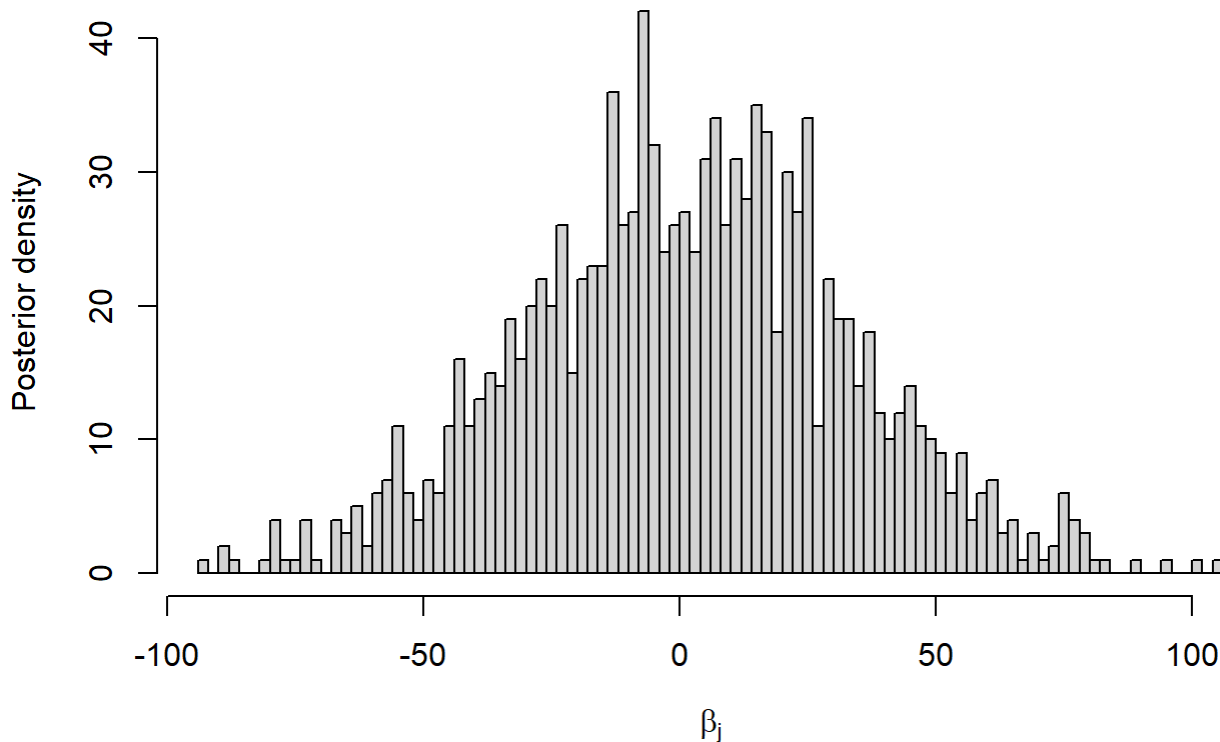
City Part Range



Is New Built



Has Storm Protector



Model 4

Set Dataframe as matrix and split dependent and independent variable

```
price <- as.matrix(df$price)
Y <- price
X <- cbind(df$squareMeters, df$numberOfRooms, df$hasYard, df$hasPool, df$floors, df$cityCode,
df$numPrevOwners, df$made, df$isNewBuilt, df$hasStormProtector, df$basement, df$attic, df$garage, df$hasStorageRoom, df$hasGuestRoom, df$cityPartRange)
names <- c("price", "Square Meters", "Number of Rooms", "Has Yard", "Has Pool", "Number of Floors", "City Code", "Number of Previous Owners", "Year Made", "is Newly Built", "Has Storm Protector", "Basement Area", "Attic Area", "Garage Area", "Has Storage Room", "Has Guest Room", "City Part Range")
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y,X=X,n=n,p=p)
params <- c("alpha","beta")
burn <- 500
n.iter <- 5000
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- theta[i] + inprod(X[i,],beta[])
  }

  # Random Effects
  for(j in 1:n){
    theta[j] ~ ddexp(0,taue)
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Set Initial Value

```
inits = list()

inits$alpha = rnorm(1)
for(i in 1:p) {
  inits$beta[i] = rnorm(1)
}
for(i in 1:n) {
  inits$theta[i] = rnorm(1)
}
inits$taue = 10
```

Compile Model

```
model4 <- jags.model(model_string, data = data, n.chains=n.chains, quiet=TRUE, inits = inits)
```

Update Model

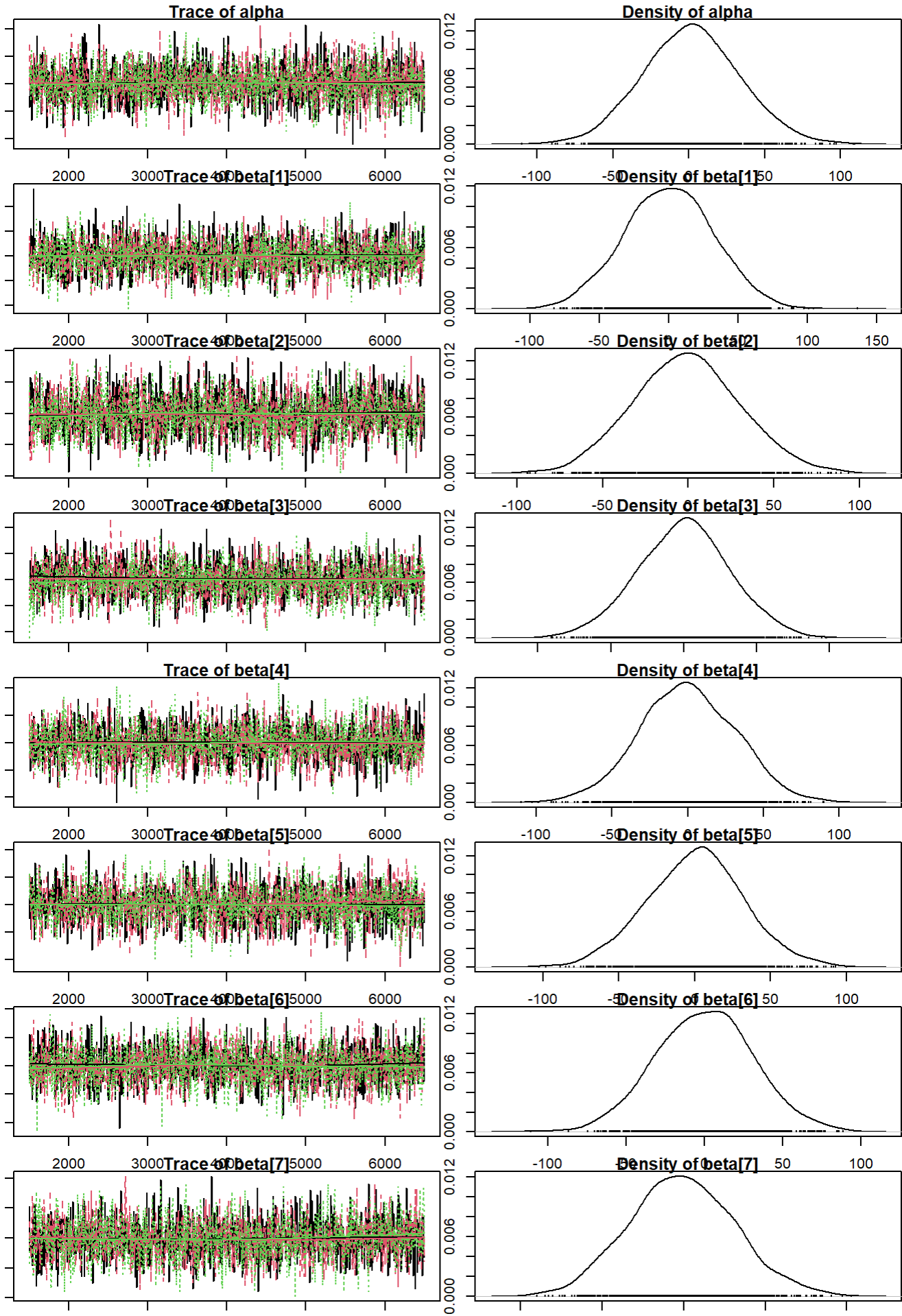
```
update(model4, burn)
```

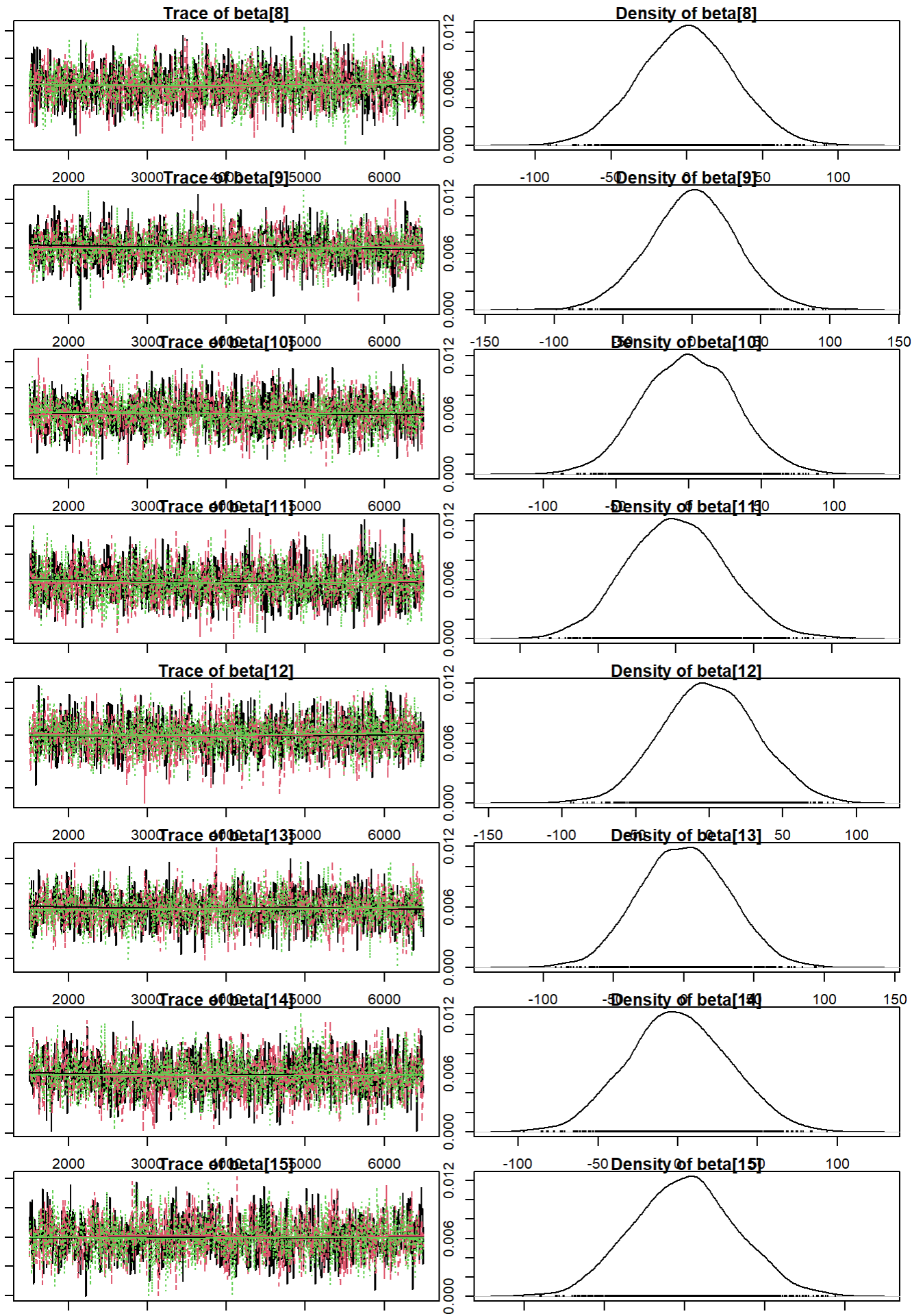
Get Posterior Samples from the model

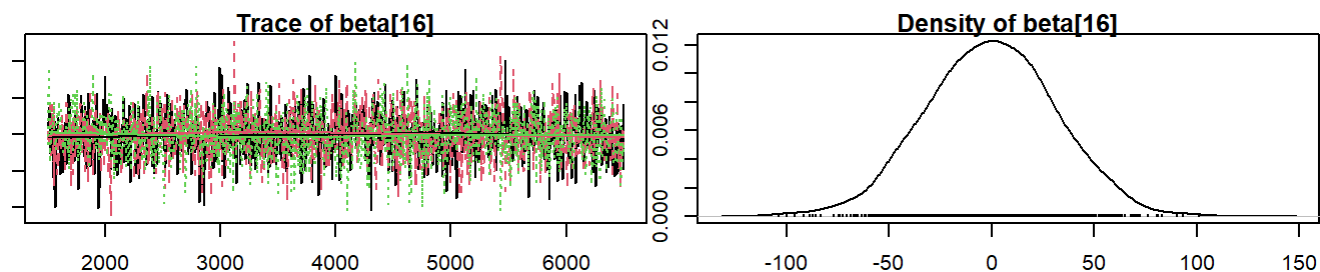
```
samples4 <- coda.samples(model4, variable.names=params, n.iter=n.iter, thin=thin)
```

Plot MCMC Chain Trace and Features Posterior Density

```
par(mar=c(1,1,1,1))
plot(samples4)
```







Show Alpha and Betas to the corresponding Features

```
sum <- summary(samples4)
rownames(sum$statistics) <- names
rownames(sum$quantiles) <- names
sum$statistics <- round(sum$statistics,3)
sum$quantiles <- round(sum$quantiles,3)
sum
```

```
##
## Iterations = 1505:6500
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 1000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##
```

	Mean	SD	Naive SE	Time-series SE
price	1.577	31.89	0.582	0.628
Square Meters	-0.069	32.12	0.586	0.652
Number of Rooms	-0.931	31.05	0.567	0.667

##	Has Yard	0.069	31.51	0.575	0.643
##	Has Pool	-0.120	31.58	0.577	0.672
##	Number of Floors	0.491	31.83	0.581	0.663
##	City Code	0.065	31.05	0.567	0.607
##	Number of Previous Owners	-1.163	32.15	0.587	0.687
##	Year Made	0.218	31.23	0.570	0.599
##	is Newly Built	0.231	31.53	0.576	0.629
##	Has Storm Protector	0.287	31.66	0.578	0.648
##	Basement Area	0.606	31.53	0.576	0.676
##	Attic Area	-0.104	32.08	0.586	0.661
##	Garage Area	0.081	31.81	0.581	0.644
##	Has Storage Room	0.253	31.64	0.578	0.672
##	Has Guest Room	-0.108	31.89	0.582	0.742
##	City Part Range	-0.464	31.79	0.580	0.655
##					
##	2. Quantiles for each variable:				
##					
##		2.5%	25%	50%	75% 97.5%
##	price	-60.26	-20.01	1.081	22.83 64.67
##	Square Meters	-62.81	-21.90	-0.184	21.68 62.55
##	Number of Rooms	-59.77	-22.00	-1.296	19.71 61.98
##	Has Yard	-63.49	-20.98	0.673	20.80 62.27
##	Has Pool	-62.99	-22.12	-0.625	21.82 61.22
##	Number of Floors	-62.89	-20.76	1.572	21.49 63.09
##	City Code	-60.29	-21.16	0.433	20.74 61.97
##	Number of Previous Owners	-62.13	-22.75	-1.598	20.75 64.16
##	Year Made	-60.47	-21.06	0.443	21.27 61.24
##	is Newly Built	-62.60	-20.54	0.539	21.44 62.08
##	Has Storm Protector	-60.11	-21.91	0.258	21.94 63.09
##	Basement Area	-60.83	-21.37	-0.044	21.66 62.14
##	Attic Area	-61.86	-21.68	-0.819	21.64 63.32
##	Garage Area	-60.88	-21.39	0.237	21.63 61.24
##	Has Storage Room	-60.07	-20.92	-0.132	21.83 62.14
##	Has Guest Room	-61.20	-21.79	0.093	21.09 62.65
##	City Part Range	-63.55	-21.75	-0.258	20.71 60.51

Check Convergence

```
gelman.diag(samples4)
```

##	Potential scale reduction factors:		
##			
##	Point est.	Upper C.I.	
##	alpha	1	1.00
##	beta[1]	1	1.00
##	beta[2]	1	1.01
##	beta[3]	1	1.00
##	beta[4]	1	1.01
##	beta[5]	1	1.01
##	beta[6]	1	1.00
##	beta[7]	1	1.01
##	beta[8]	1	1.00


```
## beta[9]          1          1.00
## beta[10]         1          1.00
## beta[11]         1          1.00
## beta[12]         1          1.00
## beta[13]         1          1.00
## beta[14]         1          1.00
## beta[15]         1          1.00
## beta[16]         1          1.00
##
## Multivariate psrf
##
## 1.01
```

Compile results

```
ESS4 <- effectiveSize(samples4)
out4 <- summary(samples4)$quantiles
rownames(out4)<-names
```

```
ESS4
```

```
##      alpha  beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
## 2637.500 2428.565 2169.397 2398.866 2236.504 2304.557 2615.739 2198.535
##  beta[8]  beta[9]  beta[10] beta[11] beta[12] beta[13] beta[14] beta[15]
## 2750.051 2517.394 2399.738 2183.856 2361.895 2465.915 2231.714 1939.749
## beta[16]
## 2355.342
```

Compute DIC & WAIC

```
# DIC
dic4 <- dic.samples(model4,n.iter=n.iter)

# WAIC
waic4 <- coda.samples(model4, variable.names=c("like"), n.iter=n.iter)
like4 <- waic4[[1]]
fbar4 <- colMeans(like4)
P4 <- sum(apply(log(like4),2,var))
WAIC4 <- -2*sum(log(fbar4))+2*P4
```

DIC and WAIC comparison

```
print("DIC Model 1:")
```

```
## [1] "DIC Model 1:"
```

```
print(dic1)
```

```
## Mean deviance: 339719
## penalty 0.9871
## Penalized deviance: 339720
```

```
print("DIC Model 2:")
```

```
## [1] "DIC Model 2:"
```

```
print(dic2)
```

```
## Mean deviance: 339719
## penalty 1.02
## Penalized deviance: 339721
```

```
print("DIC Model 3:")
```

```
## [1] "DIC Model 3:"
```

```
print(dic3)
```

```
## Mean deviance: 339719
## penalty 0.9795
## Penalized deviance: 339720
```

```
print("DIC Model 4:")
```

```
## [1] "DIC Model 4:"
```

```
print(dic4)
```

```
## Mean deviance: 182618
## penalty 10003
## Penalized deviance: 192621
```

```
print("WAIC Model 1:")
```

```
## [1] "WAIC Model 1:"
```

```
print(WAIC1)
```

```
## [1] 339719.9
```

```
print("WAIC Model 2:")
```

```
## [1] "WAIC Model 2:"
```

```
print(WAIC2)
```

```
## [1] 339719.9
```

```
print("WAIC Model 3:")
```

```
## [1] "WAIC Model 3:"
```

```
print(WAIC3)
```

```
## [1] 339719.9
```

```
print("WAIC Model 4:")
```

```
## [1] "WAIC Model 4:"
```

```
print(WAIC4)
```

```
## [1] 189549.8
```